

ЛЕКЦИЯ 9

JAVASCRIPT. ЧАСТЬ 2

ВОПРОСЫ

1. Браузерное окружение
2. DOM-дерево
3. Навигация по DOM-элементам
4. Поиск элементов
5. Свойства узлов
6. Изменение документа
7. Стили и классы
8. Браузерные события
9. Всплытие и погружение
10. Генерация событий

1

БРАУЗЕРНОЕ ОКРУЖЕНИЕ

Браузерное окружение

Окружение

Язык JavaScript изначально был создан для веб-браузеров. Но с тех пор он значительно эволюционировал и превратился в кроссплатформенный язык программирования для решения широкого круга задач.

Сегодня JavaScript может использоваться в браузере, на веб-сервере или в какой-то другой среде, даже в кофеварке. Каждая среда предоставляет свою функциональность, которую спецификация JavaScript называет окружением.

Окружение предоставляет свои объекты и дополнительные функции, в дополнение базовым языковым. Браузеры, например, дают средства для управления веб-страницами. Node.js делает доступными какие-то серверные возможности и так далее.

Браузерное окружение

Окружение

Как мы видим, имеется корневой объект window, который выступает в 2 ролях.

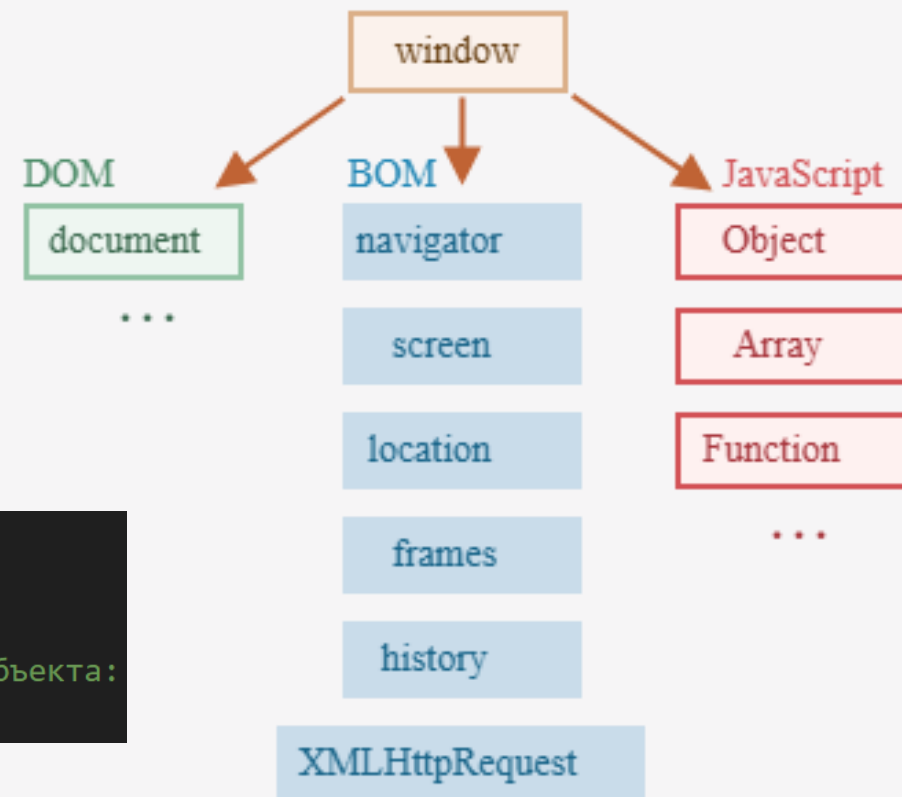
Во-первых, это глобальный объект для JavaScript-кода. Во-вторых, он также представляет собой окно браузера и располагает методами для управления им.

Например, здесь мы используем window как глобальный объект

```
function sayHi() {  
  alert("Hello");  
}  
// глобальные функции доступны как методы глобального объекта:  
window.sayHi();
```

А здесь мы используем window как объект окна браузера, чтобы узнать его высоту

```
alert(window.innerHeight); // внутренняя высота окна браузера
```



Document Object Model, сокращённо DOM - объектная модель документа, которая представляет все содержимое страницы в виде объектов, которые можно менять.

Объект `document` - основная «входная точка». С его помощью мы можем что-то создавать или менять на странице.

```
// заменим цвет фона на красный,  
document.body.style.background = "red";  
  
// а через секунду вернём как было  
setTimeout(() => document.body.style.background = "", 1000);
```

Мы использовали в примере только `document.body.style`, но на самом деле возможности по управлению страницей намного шире. Различные свойства и методы описаны в спецификации: **DOM Living Standard** на <https://dom.spec.whatwg.org>

DOM (Document Object Model)

Браузерное окружение

Спецификация DOM описывает структуру документа и предоставляет объекты для манипуляций со страницей. Существуют и другие, отличные от браузеров, инструменты, использующие DOM.

Например, серверные скрипты, которые загружают и обрабатывают HTML-страницы, также могут использовать DOM. При этом они могут поддерживать спецификацию не полностью.

Правила стилей CSS структурированы иначе чем HTML. Для них есть отдельная спецификация **CSSOM** <https://www.w3.org/TR/cssom-1/> , которая объясняет, как стили должны представляться в виде объектов, как их читать и писать.

DOM (Document Object Model)

Браузерное окружение

CSSOM используется вместе с DOM при изменении стилей документа. В реальности CSSOM требуется редко, обычно правила CSS статичны. Мы редко добавляем/удаляем стили из JavaScript, но и это возможно.

W3C Working Draft

CSS Object Model (CSSOM)

W3C Working Draft, 26 August 2021

This version:

<https://www.w3.org/TR/2021/WD-cssom-1-20210826/>

Latest published version:

<https://www.w3.org/TR/cssom-1/>

Editor's Draft:

<https://drafts.csswg.org/cssom/>

Previous Versions:

<https://www.w3.org/TR/2016/WD-cssom-1-20160317/>

<https://www.w3.org/TR/2013/WD-cssom-20131205/>

<https://www.w3.org/TR/2011/WD-cssom-20110712/>

<https://www.w3.org/TR/2000/REC-DOM-Level-2-Style-20001113/>



DOM (Document Object Model)

Браузерное окружение

BOM (Browser Object Model)

Объектная модель браузера (Browser Object Model, BOM) - это дополнительные объекты, предоставляемые браузером (окружением), чтобы работать со всем, кроме документа. Пример:

- Объект `navigator` даёт информацию о самом браузере и операционной системе. Среди множества его свойств самыми известными являются: `navigator.userAgent` - информация о текущем браузере, и `navigator.platform` - информация о платформе (может помочь в понимании того, в какой ОС открыт браузер - Windows/Linux/Mac и так далее).
- Объект `location` позволяет получить текущий URL и перенаправить браузер по новому адресу.

Вот как мы можем использовать объект `location`:

```
alert(location.href); // показывает текущий URL
if (confirm("Перейти на Wikipedia?")) {
    location.href = "https://wikipedia.org"; // перенаправляет браузер на другой URL
}
```

BOM (Browser Object Model)

Функции alert/confirm/prompt тоже являются частью BOM: они не относятся непосредственно к странице, но представляют собой методы объекта окна браузера для коммуникации с пользователем.

BOM является частью **общей спецификации HTML**: <https://html.spec.whatwg.org/>

Да, всё верно. Спецификация HTML по адресу не только про «язык HTML» (теги, атрибуты), она также покрывает целое множество объектов, методов и специфичных для каждого браузера расширений DOM. Это всё «HTML в широком смысле».

Для некоторых вещей есть отдельные спецификации, перечисленные на сайте <https://spec.whatwg.org/>



WHATWG – Standards

Standards

FAQ

Policies

Participate

The WHATWG works on a number of technologies that are fundamental parts of the web platform. They are organised somewhat arbitrarily based on the preferences of those editing the standard for those technologies.

2

ДОМ-ДЕРЕВО

Document Object Model

Основой HTML-документа являются теги.

В соответствии с объектной моделью документа ("Document Object Model", коротко DOM), каждый HTML-тег является объектом. Вложенные теги являются "детьми" родительского элемента. Текст, который находится внутри тега, также является объектом.

Все эти объекты доступны при помощи JavaScript, мы можем использовать их для изменения страницы.

Например, `document.body` - объект для тега `<body>`.

Если запустить этот код, то `<body>` станет красным на 3 секунды:

```
document.body.style.background = 'red'; // сделать фон красным  
  
setTimeout(() => document.body.style.background = '', 3000); // вернуть назад
```

Пример DOM

Начнём с такого, простого, документа:

```
<!DOCTYPE HTML>
<html>
<head>
| <title>О лосях</title>
</head>
<body>
| Правда о лосях.
</body>
</html>
```

DOM - это представление HTML-документа в виде дерева тегов. Вот как оно выглядит:

- HTML
 - HEAD
 - #text: "\n "
 - TITLE
 - #text: "О лосях"
 - #text: "\n "
 - #text: "\n "
 - BODY
 - #text: "\n Правда о правде.\n"

DOM-дерево

Пример DOM

Каждый узел этого дерева - это объект.

Теги являются **узлами-элементами** (или просто элементами). Они образуют структуру дерева: `<html>` -- это корневой узел, `<head>` и `<body>` его дочерние узлы и т.д.

Текст внутри элементов образует текстовые узлы, обозначенные как `#text`. Текстовый узел содержит в себе только строку текста. У него не может быть потомков, т.е. он находится всегда на самом нижнем уровне. Например, в теге `<title>` есть текстовый узел "О лосях".

Обратите внимание на специальные символы в текстовых узлах:

- перевод строки: `↵` (в JavaScript он обозначается как `\n`)
- пробел: `␣`

- HTML
 - HEAD
 - `#text: "\n "`
 - TITLE
 - `#text: "О лосях"`
 - `#text: "\n "`
 - `#text: "\n "`
 - BODY
 - `#text: "\n Правда о правде.\n"`

DOM-дерево

Пример DOM

- HTML
 - HEAD
 - #text: "\n "
 - TITLE
 - #text: "О лосях"
 - #text: "\n "
 - #text: "\n "
 - BODY
 - #text: "\n Правда о правде.\n"

Пробелы и переводы строки - это полноправные символы, как буквы и цифры. Они образуют текстовые узлы и становятся частью дерева DOM. Так, в примере выше в теге `<head>` есть несколько пробелов перед `<title>`, которые образуют текстовый узел `#text` (он содержит в себе только перенос строки и несколько пробелов).

Существует всего два исключения из этого правила:

- По историческим причинам пробелы и перевод строки перед тегом `<head>` игнорируются
- Если мы записываем что-либо после закрывающего тега `</body>`, браузер автоматически перемещает эту запись в конец `body`, поскольку спецификация HTML требует, чтобы всё содержимое было внутри `<body>`. Поэтому после закрывающего тега `</body>` не может быть никаких пробелов.

DOM-дерево

Пример DOM

- HTML
 - HEAD
 - TITLE
 - #text: "О лосях"
 - BODY
 - #text: "Правда о правде."

В остальных случаях всё просто - если в документе есть пробелы (или любые другие символы), они становятся текстовыми узлами дерева DOM, и если мы их удалим, то в DOM их тоже не будет.

Здесь пробельных текстовых узлов нет:

```
<!DOCTYPE HTML>  
<html><head><title>О лосях</title></head><body>Правда о лосях.</body></html>
```

Когда мы работаем с деревом DOM, используя инструменты разработчика в браузере (которые мы рассмотрим позже), пробелы в начале/конце текста и пустые текстовые узлы (переносы строк) между тегами обычно не отображаются.

Таким образом инструменты разработки экономят место на экране.

Автоисправление

Если браузер сталкивается с некорректно написанным HTML-кодом, он автоматически корректирует его при построении DOM.

Например, в начале документа всегда должен быть тег `<html>`. Даже если его нет в документе - он будет в дереве DOM, браузер его создаст. То же самое касается и тега `<body>`.

Например, если HTML-файл состоит из единственного слова "Привет", браузер обернёт его в теги `<html>` и `<body>`, добавит необходимый тег `<head>`, и DOM будет выглядеть так:

- HTML
 - HEAD
 - BODY
 - `#text: "Привет"`

При генерации DOM браузер самостоятельно обрабатывает ошибки в документе, закрывает теги и так далее.

Автоисправление

Рассмотрим пример. Есть такой документ с незакрытыми тегами:

...Но DOM будет нормальным, потому что браузер сам закроет теги и восстановит отсутствующие детали:

- HTML
 - HEAD
 - BODY
 - P
 - #text: "Привет"
 - LI
 - #text: "Мама"
 - LI
 - #text: "и"
 - LI
 - #text: "Папа"

```
<p>Привет  
<li>Мама  
<li>и  
<li>Папа
```

Автоисправление

Важный «особый случай» - работа с таблицами. По стандарту DOM у них должен быть `<tbody>`, но в HTML их можно написать (официально) без него. В этом случае браузер добавляет `<tbody>` в DOM самостоятельно.

Для такого HTML:

```
<table id="table"><tr><td>1</td></tr></table>
```

DOM-структура будет такой:

- TABLE
 - TBODY
 - TR
 - TD
 - #text: "1"

Из пустоты появился `<tbody>`, как будто документ и был таким. Важно знать об этом, иначе при работе с таблицами возможны сюрпризы.

Есть и некоторые другие типы узлов, кроме элементов и текстовых узлов.

Например, узел-комментарий:

```
<!DOCTYPE HTML>
<html>
<body>
  Правда о лосях.
  <ol>
    <li>Лось -- животное хитрое</li>
    <!-- комментарий -->
    <li>...и коварное!</li>
  </ol>
</body>
</html>
```

Другие типы узлов

DOM-дерево

- HTML
 - HEAD
 - BODY
 - #text: "\n Правда о правде.\n "
 - OL
 - #text: "\n "
 - LI
 - #text: "Правда - Это круто"
 - #text: "\n "
 - #comment: "комментарий"
 - #text: "\n "
 - LI
 - #text: "...и сила в правде!"
 - #text: "\n "
 - #text: "\n \n "

Мы видим узел нового типа - комментарий, обозначенный как `#comment`, между двумя текстовыми узлами.

Казалось бы - зачем комментарий в DOM? Он никак не влияет на визуальное отображение. Но есть важное правило: если что-то есть в HTML, то оно должно быть в DOM-дереве.

Все, что есть в HTML, даже комментарии, является частью DOM!

Даже директива `<!DOCTYPE...>`, которую мы ставим в начале HTML, тоже является DOM-узлом. Она находится в дереве DOM прямо перед `<html>`. Мы не будем рассматривать этот узел, мы даже не рисуем его на наших диаграммах, но он существует.

Даже объект `document`, представляющий весь документ, формально является DOM-узлом.

Другие типы узлов

DOM-дерево

- HTML
 - HEAD
 - BODY
 - `#text: "\n Правда о правде.\n "`
 - OL
 - `#text: "\n "`
 - LI
 - `#text: "Правда - Это круто"`
 - `#text: "\n "`
 - `#comment: "комментарий"`
 - `#text: "\n "`
 - LI
 - `#text: "...и сила в правде!"`
 - `#text: "\n "`
 - `#text: "\n \n"`

Существует 12 типов узлов. Но на практике мы в основном работаем с 4 из них:

- `document` - "входная точка" в DOM.
- узлы-элементы - HTML-теги, основные строительные блоки.
- текстовые узлы - содержат текст.
- комментарии - иногда в них можно включить информацию, которая не будет показана, но доступна в DOM для чтения JS.

Подробнее про **другие узлы** тут:

<https://dom.spec.whatwg.org/#node>

DOM

**Living Standard — Last Updated 15
January 2026**



Другие типы узлов

DOM-дерево

Взаимодействие с консолью

При работе с DOM нам часто требуется применить к нему JavaScript. Например: получить узел и запустить какой-нибудь код для его изменения, чтобы посмотреть результат. Вот несколько подсказок, как перемещаться между вкладками Elements и Console.

Для начала:

1. На вкладке Elements выберите первый элемент `<li>`.
2. Нажмите `key:Esc` -- прямо под вкладкой Elements откроется Console.

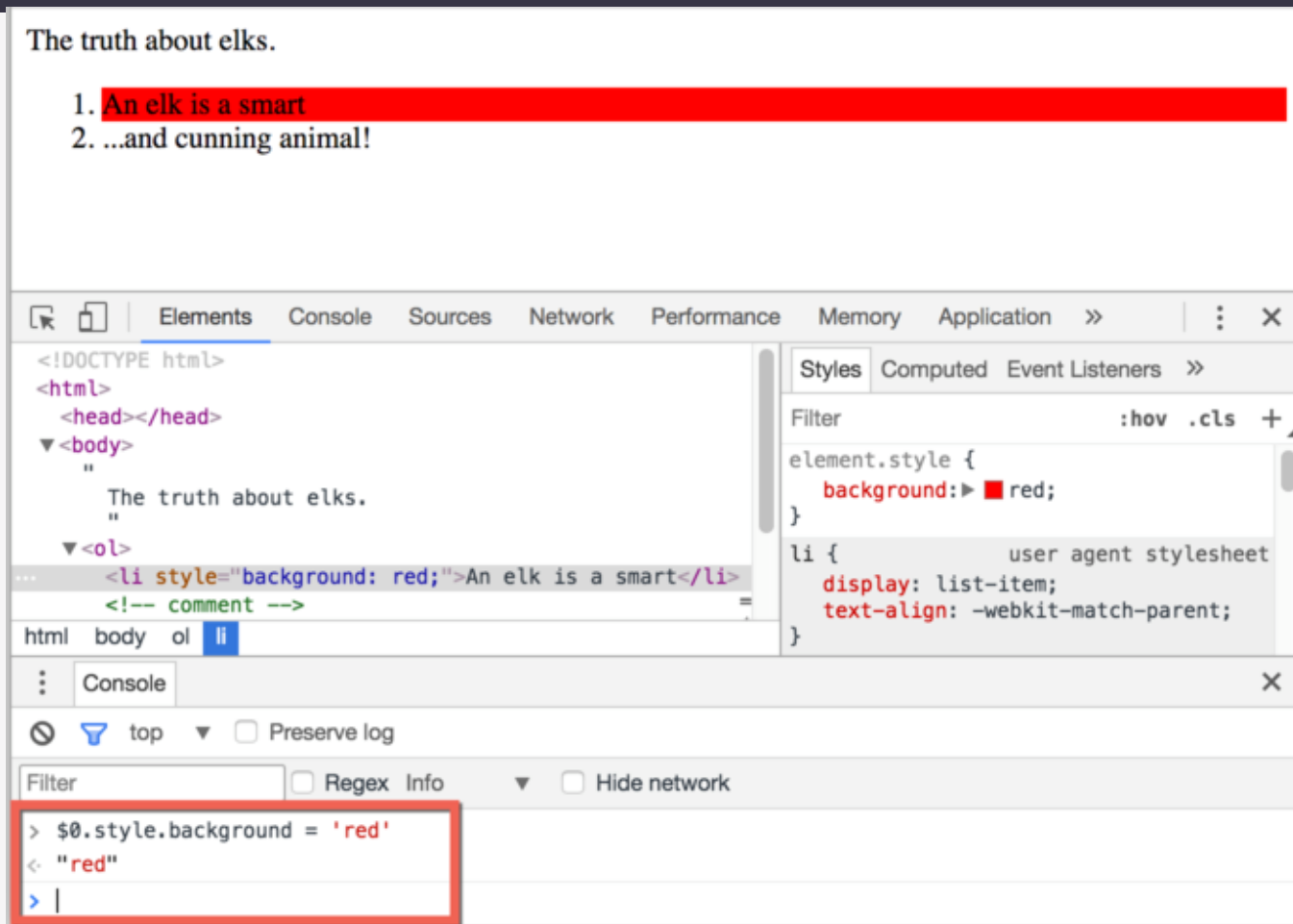
Последний элемент, выбранный во вкладке Elements, доступен в консоли как `$0`; предыдущий, выбранный до него, как `$1` и т.д.

DOM-дерево

Взаимодействие с консолью

Теперь мы можем запускать на них команды. Например `$0.style.background = 'red'` сделает выбранный элемент красным, как здесь:

Это мы посмотрели как получить узел из Elements в Console.



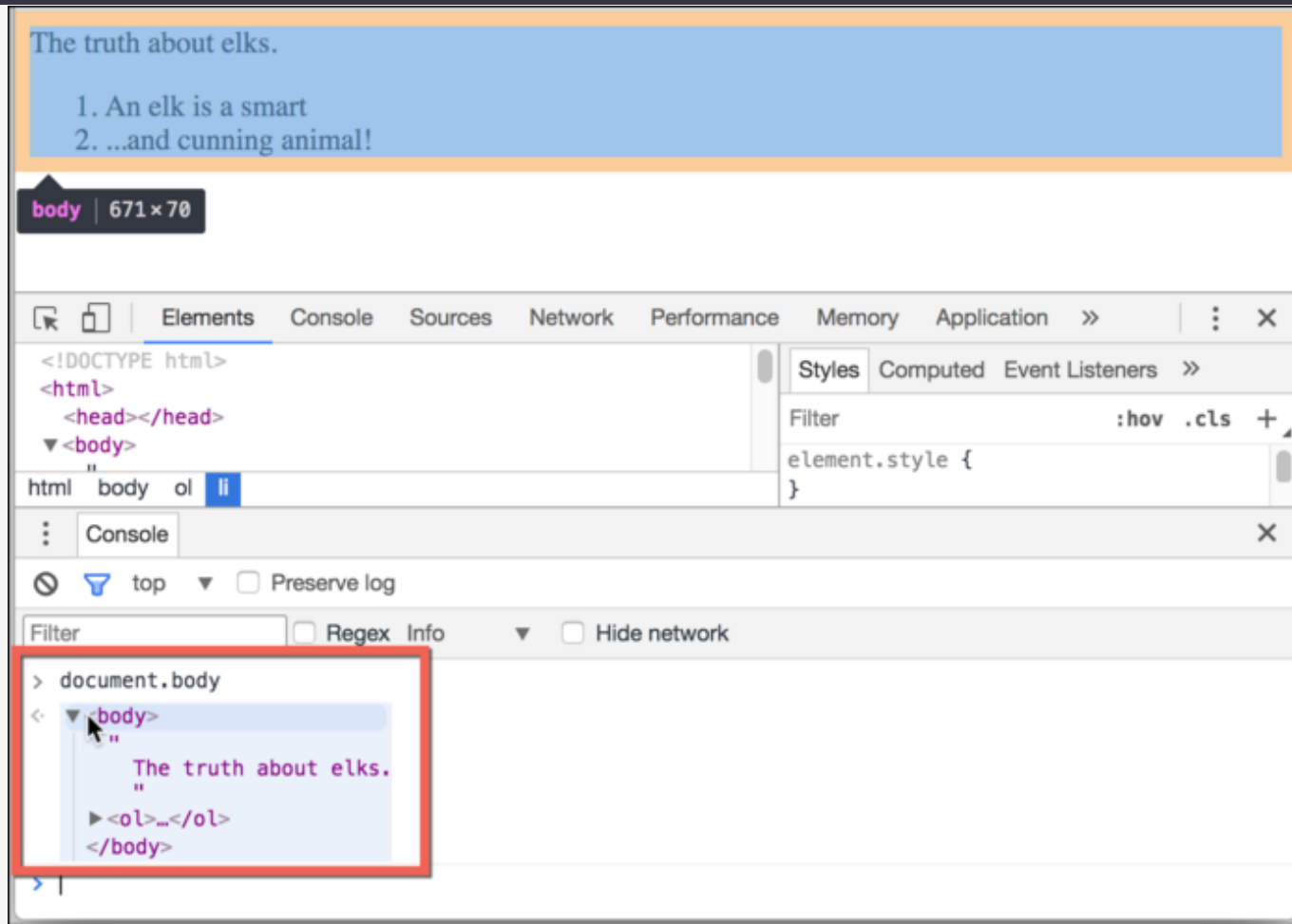
DOM-дерево

Взаимодействие с консолью

Есть и обратный путь: если есть переменная `node`, ссылающаяся на DOM-узел, можно использовать в консоли команду `inspect(node)`, чтобы увидеть этот элемент во вкладке Elements.

Или мы можем просто вывести DOM-узел в консоль и исследовать «на месте», как `document.body` ->

Это может быть полезно для отладки.



3

НАВИГАЦИЯ ПО DOM-ЭЛЕМЕНТАМ

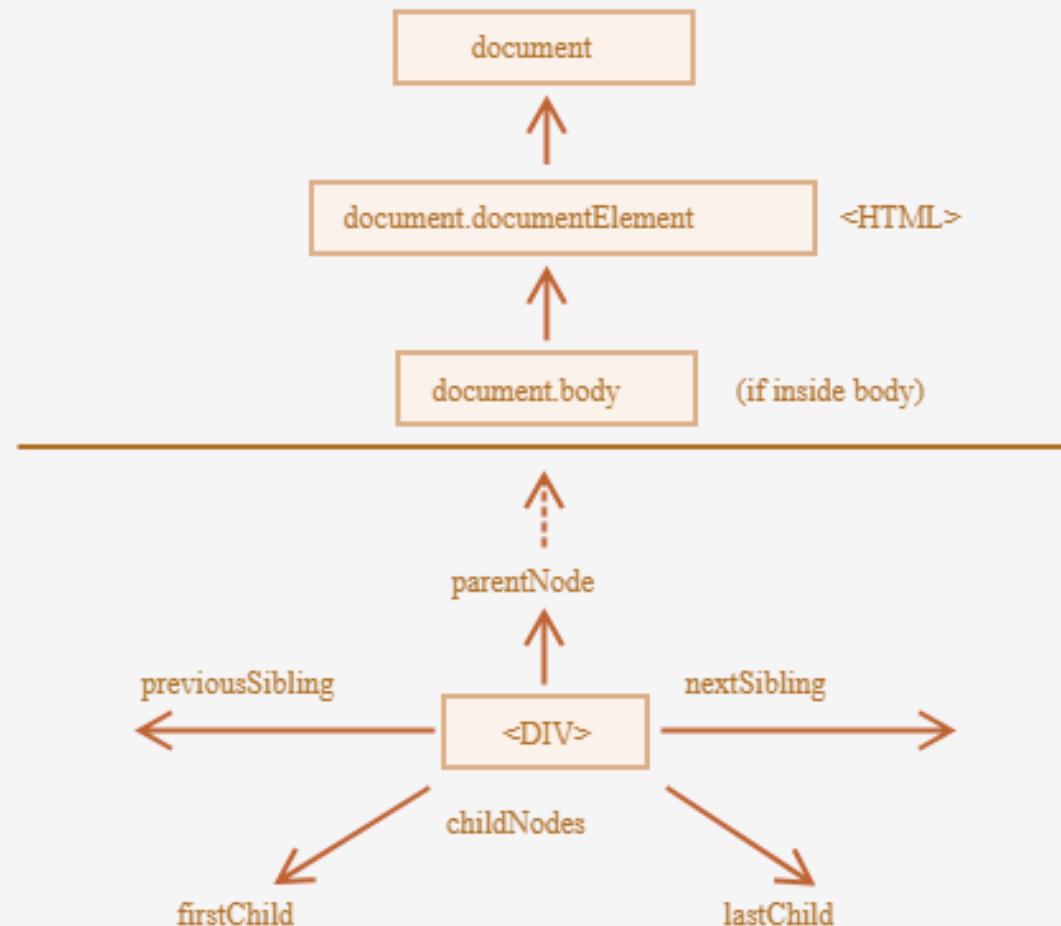
Навигация по DOM-элементам

DOM-элементы

DOM позволяет нам делать что угодно с элементами и их содержимым, но для начала нужно получить соответствующий DOM-объект.

Все операции с DOM начинаются с объекта `document`. Это главная «точка входа» в DOM. Из него мы можем получить доступ к любому узлу.

Так выглядят основные ссылки, по которым можно переходить между узлами DOM



Навигация по DOM-элементам

documentElement и body

Самые верхние элементы дерева доступны как свойства объекта `document`:

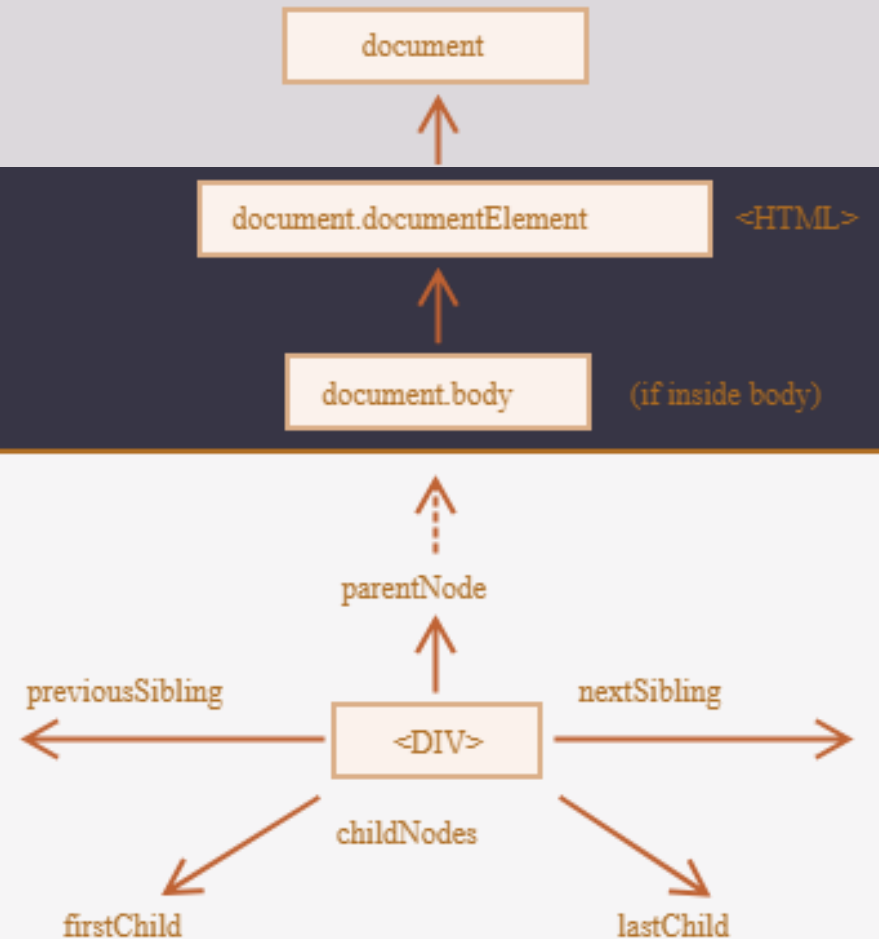
`<html> = document.documentElement`:

Самый верхний узел документа:

`document.documentElement`. В DOM он соответствует тегу `<html>`.

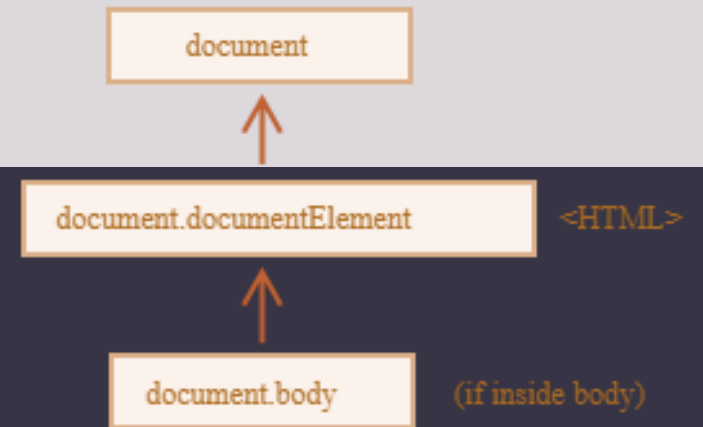
`<body> = document.body`: Другой часто используемый DOM-узел - узел тега `<body>`: `document.body`.

`<head> = document.head`: Тег `<head>` доступен как `document.head`.



Навигация по DOM-элементам

documentElement и body



Нельзя получить доступ к элементу, которого ещё не существует в момент выполнения скрипта.

Сообщение с 127.0.0.1:5500:

Из BODY: [object HTMLBodyElement]

OK

Сообщение с 127.0.0.1:5500:

Из HEAD: null

OK

head>, document.body в нём недоступен, потому что

```
html>
head>
<script>
  alert( "Из HEAD: " + document.body ); // null, <body> ещё нет
</script>
head>
body>
<script>
  alert( "Из BODY: " + document.body ); // HTMLBodyElement, теперь он есть
</script>
body>
html>
```

Навигация по DOM-элементам

documentElement и body

```
<html>
<head>
  <script>
    alert( "Из HEAD: " + document.body ); // null, <body> ещё нет
  </script>
</head>
<body>
  <script>
    alert( "Из BODY: " + document.body ); // HTMLBodyElement, теперь он есть
  </script>
</body>
</html>
```

Сообщение с 127.0.0.1:5500:

Из BODY: [object HTMLBodyElement]

OK

Сообщение с 127.0.0.1:5500:

Из HEAD: null

OK

Здесь и далее мы будем использовать два принципиально разных термина:

- **Дочерние узлы (или дети)** - элементы, которые являются непосредственными детьми узла. Другими словами, элементы, которые лежат непосредственно внутри данного. Например, `<head>` и `<body>` являются детьми элемента `<html>`.
- **Потомки** - все элементы, которые лежат внутри данного, включая детей, их детей и т.д.

В примере детьми тега `<body>` являются теги `<div>` и `<ul>` (и несколько пустых текстовых узлов):

...А потомки `<body>`- это и прямые дети `<div>`, `<ul>`
и вложенные в них: `<li>` (ребёнок `<ul>`) и `<b>` (ребёнок `<li>`)
- в общем, все элементы поддерева.

```
<html>
<body>
  <div>Начало</div>
  <ul>
    <li>
      <b>Информация</b>
    </li>
  </ul>
</body>
</html>
```

Дети: `childNodes`, `firstChild`, `lastChild`

Навигация по DOM-элементам

Коллекция `childNodes` содержит список всех детей, включая текстовые узлы.
Пример последовательно выведет детей `document.body`

```
<html>
<body>
  <div>Начало</div>
  <ul>
    <li>Информация</li>
  </ul>
</div>Конец</div>

  <script>
    for (let i = 0; i < document.body.childNodes.length; i++) {
      alert( document.body.childNodes[i] ); // Text, DIV, Text, UL, ..., SCRIPT
    }
  </script>
  ...какой-то HTML-код...
</body>
</html>
```

Сообщение с 127.0.0.1:5500:

[object HTMLDivElement]

OK

Сообщение с 127.0.0.1:5500:

[object Text]

Сообщение с 127.0.0.1:5500:

[object HTMLULListElement]

Сообщение с 127.0.0.1:5500:

[object Text]

Дети: `childNodes`, `firstChild`, `lastChild`

Навигация по DOM-элементам

```
<html>
<body>
  <div>Начало</div>
  <ul>
    <li>Информация</li>
  </ul>
  <div>Конец</div>

  <script>
    for (let i = 0; i < document.body.childNodes.length; i++) {
      alert( document.body.childNodes[i] ); // Text, DIV, Text, UL, ..., SCRIPT
    }
  </script>
  ...какой-то HTML-код...
</body>
</html>
```

Сообщение с 127.0.0.1:5500:

[object HTMLScriptElement]

OK

Обратим внимание на маленькую деталь. Если запустить пример, то последним будет выведен элемент `<script>`. На самом деле, в документе есть ещё «какой-то HTML-код», но на момент выполнения скрипта браузер ещё до него не дошёл, поэтому скрипт не видит его.

Дети: `childNodes`, `firstChild`, `lastChild`

Навигация по DOM-элементам

Свойства `firstChild` и `lastChild` обеспечивают быстрый доступ к первому и последнему дочернему элементу.

Они, по сути, являются всего лишь сокращениями.

Если у тега есть дочерние узлы, условие ниже всегда верно:

```
elem.childNodes[0] === elem.firstChild  
elem.childNodes[elem.childNodes.length - 1] === elem.lastChild
```

Для проверки наличия дочерних узлов существует также специальная функция `elem.hasChildNodes()`.

Дети: `childNodes`, `firstChild`, `lastChild`

Навигация по DOM-элементам

DOM-коллекции

Как мы уже видели, `childNodes` похож на массив. На самом деле это не массив, а коллекция - особый перебираемый объект-псевдомассив.

И есть два важных следствия из этого:

1

Для перебора **коллекции** мы можем использовать `for..of`

```
for (let node of document.body.childNodes) {  
  alert(node); // покажет все узлы из коллекции  
}
```

Это работает, потому что коллекция является перебираемым объектом (есть требуемый для этого метод `Symbol.iterator`).

DOM-коллекции

2

Методы массивов не будут работать, потому что коллекция - это не массив

```
alert(document.body.childNodes.filter); // undefined (у коллекции нет метода filter!)
```

Первый пункт - это хорошо для нас. Второй - бывает неудобен, но можно пережить.

Если нам хочется использовать именно методы массива, то мы можем создать настоящий массив из коллекции, используя `Array.from`

```
alert( Array.from(document.body.childNodes).filter ); // сделали массив
```

DOM-коллекции, и даже более - все перечисленные навигационные свойства, доступны только для чтения. Мы не можем заменить один дочерний узел на другой, просто написав `childNodes[i] = ....`. Для изменения DOM требуются другие методы.

Навигация по DOM-элементам

DOM-коллекции

Почти все DOM-коллекции, за небольшим исключением, живые. Другими словами, они отражают текущее состояние DOM.

Если мы сохраним ссылку на `elem.childNodes` и добавим/удалим узлы в DOM, то они появятся в сохранённой коллекции автоматически.

Коллекции перебираются циклом `for..of`.

Некоторые начинающие разработчики пытаются использовать для этого цикл `for..in`.

Не делайте так. Цикл `for..in` перебирает все перечисляемые свойства. А у коллекций есть некоторые «лишние», редко используемые свойства, которые обычно нам не нужны

```
<body>
<script>
  // выводит 0, 1, length, item, values и другие свойства.
  for (let prop in document.body.childNodes) alert(prop);
</script>
</body>
```

Соседи - это узлы, у которых один и тот же родитель.
Например, здесь `<head>` и `<body>` соседи:

- говорят, что `<body>` - «следующий» или «правый» сосед `<head>`
- также можно сказать, что `<head>` «предыдущий» или «левый» сосед `<body>`

Следующий узел того же родителя (следующий сосед) - доступен в свойстве `nextSibling`, а предыдущий - в `previousSibling`.

Родитель доступен через `parentNode`.

Например:

```
<html>
|  <head>...</head><body>...</body>
|</html>
```

```
// родителем <body> является <html>
alert( document.body.parentNode === document.documentElement ); // выведет true

// после <head> идёт <body>
alert( document.head.nextSibling ); // HTMLBodyElement

// перед <body> находится <head>
alert( document.body.previousSibling ); // HTMLHeadElement
```

Соседи и родитель

Навигация по DOM-элементам

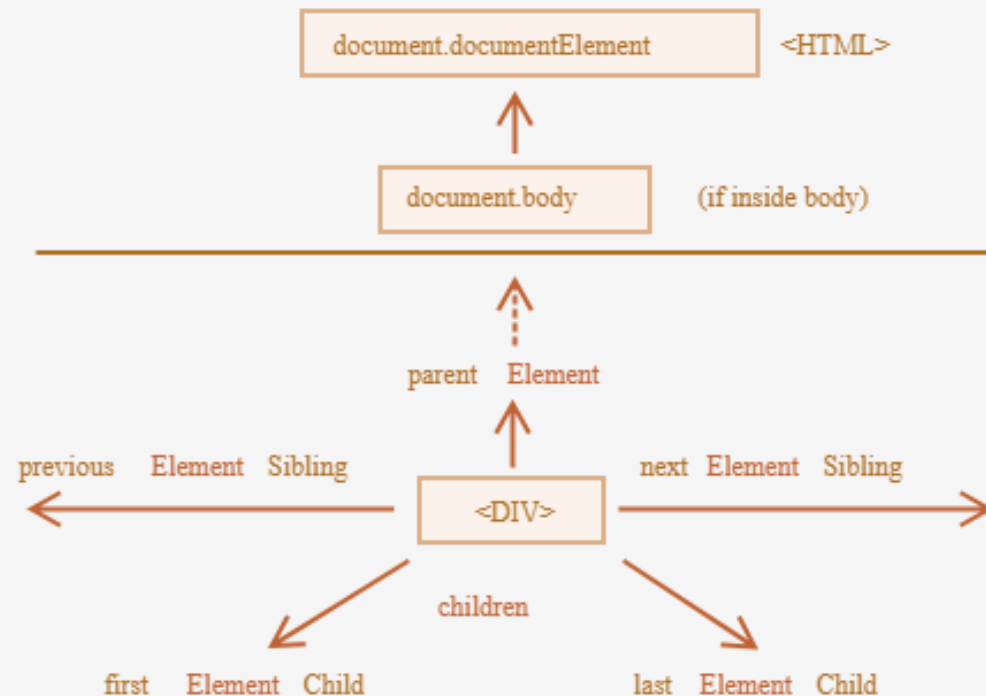
Навигация по DOM-элементам

Навигация только по элементам

Навигационные свойства, описанные выше, относятся ко всем узлам в документе. В частности, в `childNodes` находятся и текстовые узлы и узлы-элементы и узлы-комментарии, если они есть.

Но для большинства задач текстовые узлы и узлы-комментарии нам не нужны. Мы хотим манипулировать узлами-элементами, которые представляют собой теги и формируют структуру страницы.

Поэтому давайте рассмотрим дополнительный набор ссылок, которые учитывают только **узлы-элементы**:

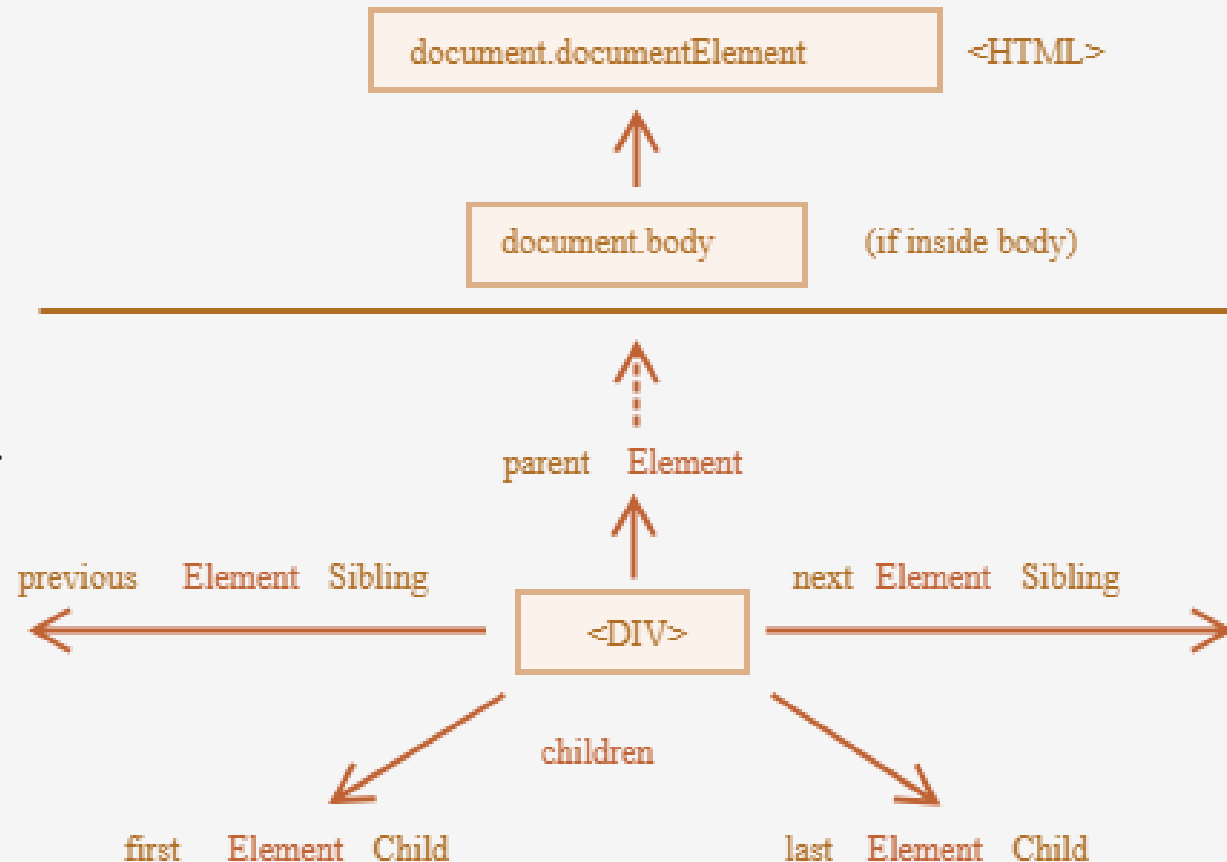


Навигация по DOM-элементам

Навигация только по элементам

Эти ссылки похожи на те, что раньше, только в ряде мест стоит слово Element:

- **children** - коллекция детей, которые являются элементами.
- **firstElementChild, lastElementChild** - первый и последний дочерний элемент.
- **previousElementSibling, nextElementSibling** - соседи-элементы.
- **parentElement** - родитель-элемент.



Навигация по DOM-элементам

Навигация только по элементам

Свойство `parentElement` возвращает родитель-элемент, а `parentNode` возвращает «любого родителя». Обычно эти свойства одинаковы: они оба получают родителя.

За исключением

`document.documentElement`:

```
alert( document.documentElement.parentNode ); // выведет document
alert( document.documentElement.parentElement ); // выведет null
```

Причина в том, что родителем корневого узла `document.documentElement` (`<html>`) является `document`. Но `document` - это не узел-элемент, так что `parentNode` вернёт его, а `parentElement` нет.

Эта деталь может быть полезна, если хотим пройти вверх по цепочке родителей от произвольного элемента `elem` к `<html>`, но не до `document`:

```
while(elem = elem.parentElement) { // идти наверх до <html>
  alert( elem );
}
```

Навигация по DOM-элементам

Навигация только по элементам

Изменим один из примеров выше.
Заменяем `childNodes` на `children`.

Теперь цикл выводит только элементы:

Сообщение с 127.0.0.1:5500:

[object HTMLDivElement]

Сообщение с 127.0.0.1:5500:

[object HTMLUListElement]

Сообщение с 127.0.0.1:5500:

[object HTMLScriptElement]

```
<html>
<body>
  <div>Начало</div>

  <ul>
    <li>Информация</li>
  </ul>

  <div>Конец</div>

  <script>
    for (let elem of document.body.children) {
      alert(elem); // DIV, UL, DIV, SCRIPT
    }
  </script>
  ...
</body>
</html>
```

4

ПОИСК ЭЛЕМЕНТОВ

document.getElementById или просто id

Свойства навигации по DOM хороши, когда элементы расположены рядом. А если нет? Как получить произвольный элемент страницы? Для этого в DOM есть дополнительные методы поиска.

Если у элемента есть атрибут id, то мы можем получить его вызовом document.getElementById(id), где бы он ни находился. Например:

```
<div id="elem">
  <div id="elem-content">Element</div>
</div>

<script>
  // получить элемент
  let elem = document.getElementById('elem');

  // сделать его фон красным
  elem.style.background = 'red';
</script>
```

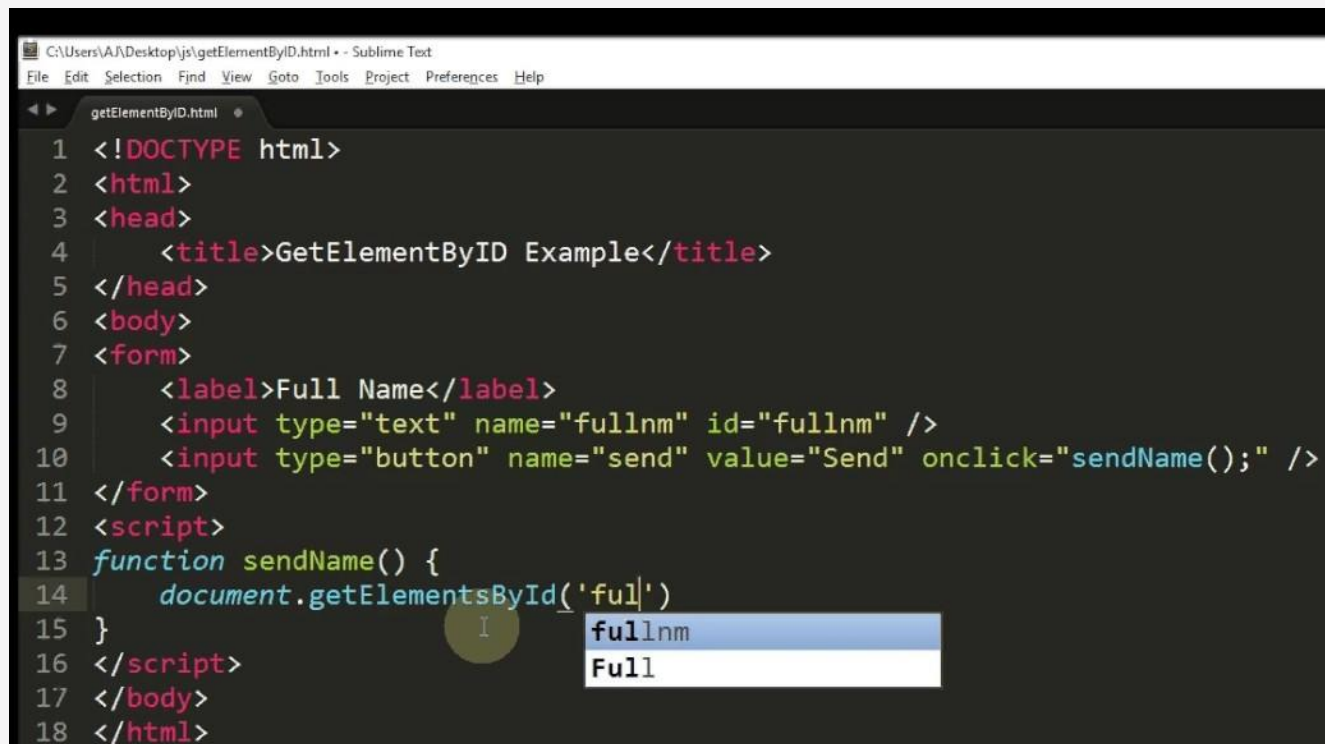
document.getElementById или просто id

Значение id должно быть уникальным. В документе может быть только один элемент с данным id.

Если в документе есть несколько элементов с одинаковым значением id, то поведение методов поиска непредсказуемо. Браузер может вернуть любой из них случайным образом. Поэтому, придерживайтесь правила сохранения уникальности id.

Метод getElementById можно вызвать только для объекта document.

Он осуществляет поиск по id по всему документу.



```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>GetElementById Example</title>
5 </head>
6 <body>
7 <form>
8   <label>Full Name</label>
9   <input type="text" name="fullnm" id="fullnm" />
10  <input type="button" name="send" value="Send" onclick="sendName();" />
11 </form>
12 <script>
13 function sendName() {
14   document.getElementById('fullnm')
15 }
16 </script>
17 </body>
18 </html>
```

Самый универсальный метод поиска - это `elem.querySelectorAll(css)`, он возвращает все элементы внутри `elem`, удовлетворяющие данному CSS-селектору.

Следующий запрос получает все элементы `<li>`, которые являются последними потомками в `<ul>`

Этот метод действительно мощный, потому что можно использовать любой CSS-селектор.

Псевдоклассы в CSS-селекторе, в частности `:hover` и `:active`, также поддерживаются.

Например, `document.querySelectorAll(':hover')` вернёт коллекцию

(в порядке вложенности: от внешнего к внутреннему) из текущих элементов под курсором мыши.

querySelectorAll

Поиск элементов

```
<ul>
  <li>Этот</li>
  <li>тест</li>
</ul>
<ul>
  <li>полностью</li>
  <li>пройден</li>
</ul>
<script>
  let elements = document.querySelectorAll('ul > li:last-child');

  for (let elem of elements) {
    alert(elem.innerHTML); // "тест", "пройден"
  }
</script>
```

Метод `elem.querySelector(css)` возвращает первый элемент, соответствующий данному CSS-селектору.

Иначе говоря, результат такой же, как при вызове `elem.querySelectorAll(css)[0]`, но он сначала найдёт все элементы, а потом возьмёт первый, в то время как `elem.querySelector` найдёт только первый и остановится. Это быстрее, кроме того, его короче писать.

```
<!DOCTYPE html>
<html>
<body>

<h2 class="Selector">A heading with class name as "Selector"</h2>
<p class="Selector">A paragraph with class name as "Selector".</p>

<button onclick="myFunction()">Click here</button>

<script>
function myFunction() {
    document.querySelector(".Selector").style.backgroundColor = "Blue";
}
</script>

</body>
</html>
```

querySelector

Поиск элементов

Поиск элементов

matches

Предыдущие методы искали по DOM.

Метод `elem.matches(css)` ничего не ищет, а проверяет, удовлетворяет ли `elem` CSS-селектору, и возвращает `true` или `false`.

Этот метод удобен, когда мы перебираем элементы (например, в массиве или в чём-то подобном) и пытаемся выбрать те из них, которые нас интересуют.

Например:

```
<a href="http://example.com/file.zip">...</a>
<a href="http://ya.ru">...</a>

<script>
  // может быть любая коллекция вместо document.body.children
  for (let elem of document.body.children) {
    if (elem.matches('a[href$="zip"]')) {
      alert("Ссылка на архив: " + elem.href );
    }
  }
</script>
```

Поиск элементов

closest

Предки элемента - родитель, родитель родителя, его родитель и так далее. Вместе они образуют цепочку иерархии от элемента до вершины.

Метод `elem.closest(css)` ищет ближайшего предка, который соответствует CSS-селектору. Сам элемент также включается в поиск.

Другими словами, метод `closest` поднимается вверх от элемента и проверяет каждого из родителей. Если он соответствует селектору, поиск прекращается. Метод возвращает либо предка, либо `null`, если такой элемент не найден.

Пример представлен выше.

```
<h1>Содержание</h1>
```

```
<div class="contents">
```

```
  <ul class="book">
```

```
    <li class="chapter">Глава 1</li>
```

```
    <li class="chapter">Глава 2</li>
```

```
  </ul>
```

```
</div>
```

```
<script>
```

```
  let chapter = document.querySelector('.chapter'); // LI
```

```
  alert(chapter.closest('.book')); // UL
```

```
  alert(chapter.closest('.contents')); // DIV
```

```
  alert(chapter.closest('h1')); // null (потому что h1 - не предок)
```

```
</script>
```

Существуют также другие методы поиска элементов по тегу, классу и так далее. На данный момент, они скорее исторические, так как `querySelector` более чем эффективен.

Рассмотрим их для полноты картины, также вы можете встретить их в старом коде.

- `elem.getElementsByTagName(tag)` ищет элементы с данным тегом и возвращает их коллекцию. Передав "*" вместо тега, можно получить всех потомков.
- `elem.getElementsByTagName(className)` возвращает элементы, которые имеют данный CSS-класс.
- `document.getElementsByTagName(name)` возвращает элементы с заданным атрибутом `name`. Очень редко используется.

Например:

```
// получить все элементы div в документе  
let divs = document.getElementsByTagName('div');
```

getElementsByTagName

Поиск элементов

Давайте найдём
все input в таблице:

getElementsByTagName

Поиск элементов

```
<table id="table">
  <tr>
    <td>Ваш возраст:</td>

    <td>
      <label>
        <input type="radio" name="age" value="young" checked> младше 18
      </label>
      <label>
        <input type="radio" name="age" value="mature"> от 18 до 50
      </label>
      <label>
        <input type="radio" name="age" value="senior"> старше 60
      </label>
    </td>
  </tr>
</table>

<script>
  let inputs = table.getElementsByTagName('input');

  for (let input of inputs) {
    alert( input.value + ': ' + input.checked );
  }
</script>
```

Одна из самых частых ошибок начинающих разработчиков (впрочем, иногда и не только) - это забыть букву "s". То есть пробовать вызывать метод `getElementByTagName` вместо `getElementsByTagName`.

Буква "s" отсутствует в названии метода `getElementById`, так как в данном случае возвращает один элемент. Но `getElementsByTagName` вернёт список элементов, поэтому "s" обязательна.

Попытка присвоить значение коллекции, а не элементам внутри неё, не сработает.

```
// не работает  
document.getElementsByTagName('input').value = 5;
```

Нужно перебрать коллекцию в цикле или получить элемент по номеру и уже ему присваивать значение, например, так:

```
// работает (если есть input)  
document.getElementsByTagName('input')[0].value = 5;
```

getElementsBy

Поиск элементов

Ищем элементы с классом .article:

```
<form name="my-form">
  <div class="article">Article</div>
  <div class="long article">Long article</div>
</form>

<script>
  // ищем по имени атрибута
  let form = document.getElementsByName('my-form')[0];

  // ищем по классу внутри form
  let articles = form.getElementsByClassName('article');
  alert(articles.length); // 2, находим два элемента с классом article
</script>
```

getElementsBy

Поиск элементов

Живые коллекции

Все методы "getElementsBy*" возвращают живую коллекцию. Такие коллекции всегда отражают текущее состояние документа и автоматически обновляются при его изменении.

В приведённом примере есть два скрипта.

1. Первый создаёт ссылку на коллекцию <div>. На этот момент её длина равна 1.
2. Второй скрипт запускается после того, как браузер встречает ещё один <div>, теперь её длина - 2.

```
<div>First div</div>
```

```
<script>
```

```
  let divs = document.getElementsByTagName('div');
```

```
  alert(divs.length); // 1
```

```
</script>
```

```
<div>Second div</div>
```

```
<script>
```

```
  alert(divs.length); // 2
```

```
</script>
```

Поиск элементов

Живые коллекции

Напротив, `querySelectorAll` возвращает статическую коллекцию. Это похоже на фиксированный массив элементов.

Если мы будем использовать его в примере выше, то оба скрипта вернут длину коллекции, равную 1:

Теперь мы легко видим разницу. Длина статической коллекции не изменилась после появления нового `div` в документе.

```
<div>First div</div>
```

```
<script>  
  let divs = document.querySelectorAll('div');  
  alert(divs.length); // 1  
</script>
```

```
<div>Second div</div>
```

```
<script>  
  alert(divs.length); // 1  
</script>
```

Поиск элементов

Итого

```
1 // Searching elements methods
2 document.getElementById('id');
3 document.getElementsByName('name');
4 document.getElementsByTagName('div');
5 document.getElementsByClassName('class');
6 document.querySelector('nav + p');
7 document.querySelectorAll('nav > ul > li');
```

Метод	Поиск по	Ищет внутри элемента?	Что возвращает?	Возвращает живой набор элементов?
querySelector	CSS-селектору	✓	HTMLElement или null	
querySelectorAll	CSS-селектору	✓	NodeList	
getElementById	id		HTMLElement или null	
getElementsByName	атрибуту name		NodeList	✓
getElementsByTagName	тегу или *	✓	HTMLCollection	✓
getElementsByClassName	классу	✓	HTMLCollection	✓

5

СВОЙСТВА УЗЛОВ

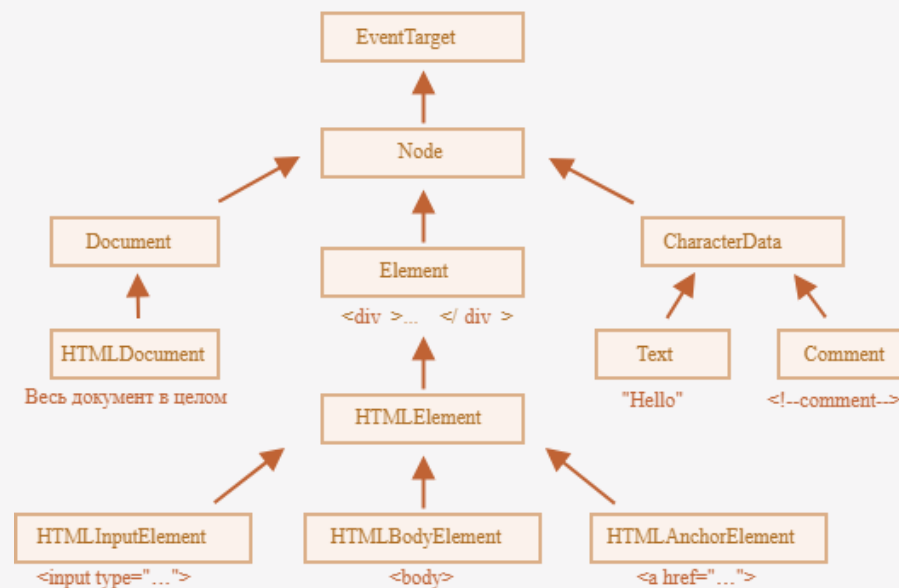
Классы DOM-узлов

У разных DOM-узлов могут быть разные свойства. Например, у узла, соответствующего тегу `<a>`, есть свойства, связанные со ссылками, а у соответствующего тегу `<input>` - свойства, связанные с полем ввода и т.д. Текстовые узлы отличаются от узлов-элементов. Но у них есть общие свойства и методы, потому что все классы DOM-узлов образуют единую иерархию.

Каждый DOM-узел принадлежит соответствующему встроенному классу.

Корнем иерархии является `EventTarget`, от него наследует `Node` и остальные DOM-узлы.

На рисунке ниже изображены основные классы

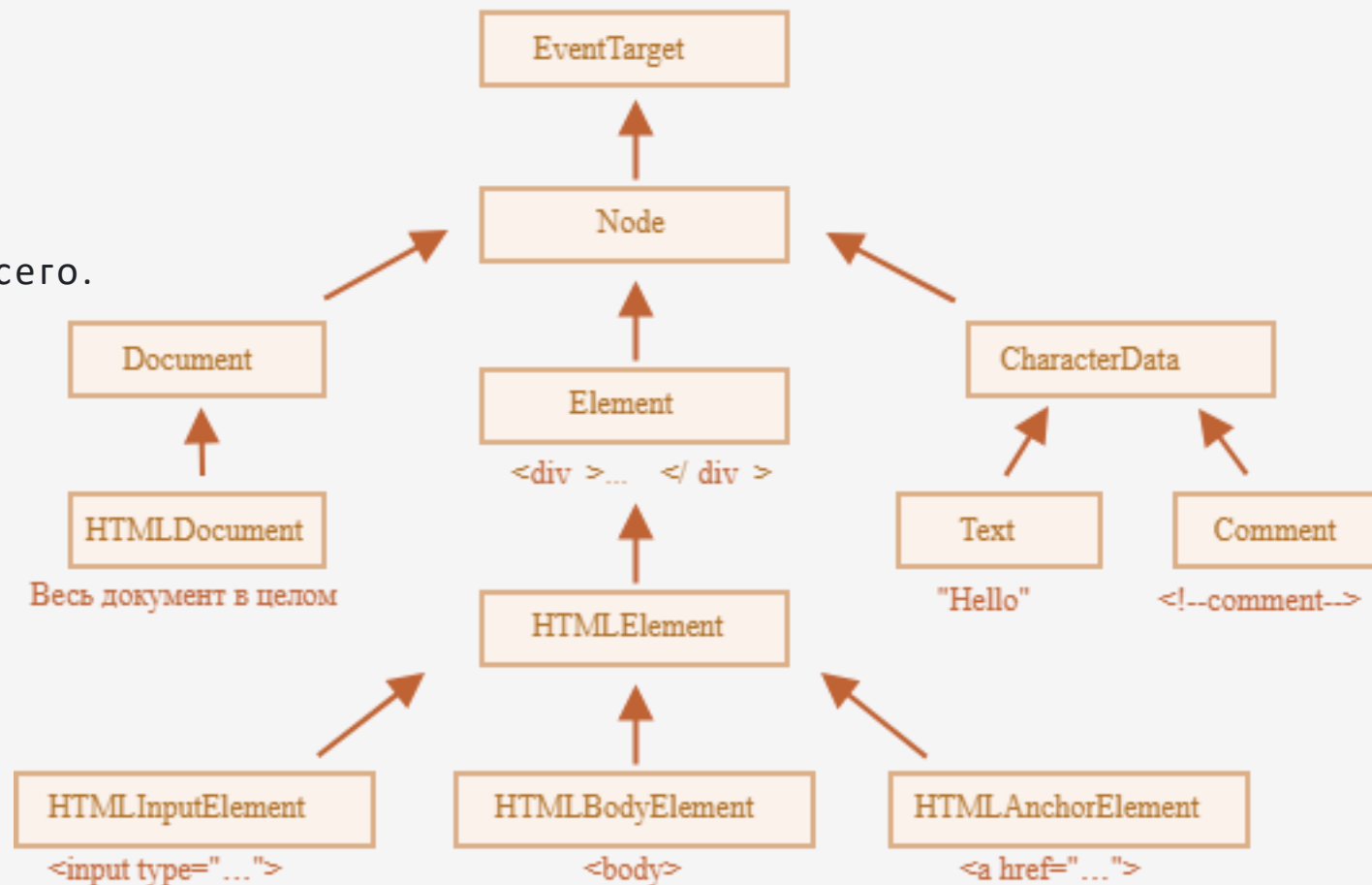


Классы DOM-узлов

Существуют следующие классы:

1 **EventTarget** - это корневой «абстрактный» класс для всего.

Объекты этого класса никогда не создаются. Он служит основой, благодаря которой все DOM-узлы поддерживают так называемые «события», о которых поговорим позже.



Классы DOM-узлов

2 Node - также является «абстрактным» классом, и служит основой для DOM-узлов.

Он обеспечивает базовую функциональность: `parentNode`, `nextSibling`, `childNodes` и т.д. (это геттеры). Объекты класса `Node` никогда не создаются. Но есть определённые классы узлов, которые наследуются от него (и следовательно наследуют функционал `Node`).

3 Document, по историческим причинам часто наследуется `HTMLDocument` (хотя последняя спецификация этого не навязывает) - это документ в целом.

Глобальный объект `document` принадлежит именно к этому классу. Он служит точкой входа в DOM.

4 CharacterData – «абстрактный» класс. Вот, кем он наследуется:

- `Text` - класс, соответствующий тексту внутри элементов. Например, `Hello` в `<p>Hello</p>`.
- `Comment` - класс для комментариев. Они не отображаются, но каждый комментарий становится членом DOM.

Классы DOM-узлов

5 **Element** - это базовый класс для DOM-элементов.

Он обеспечивает навигацию на уровне элементов: `nextElementSibling`, `children`. А также и методы поиска элементов: `getElementsByTagName`, `querySelector`.

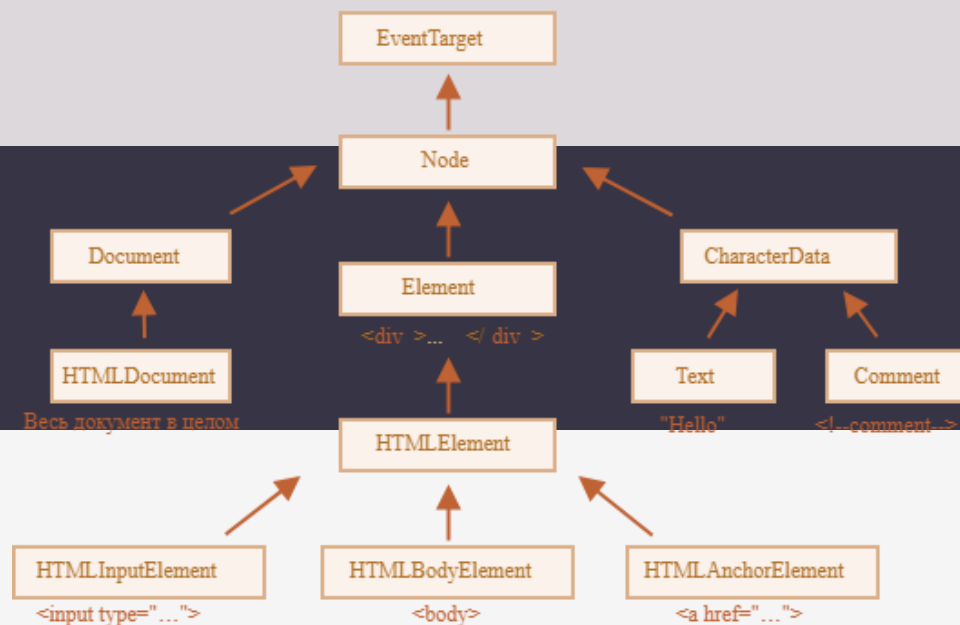
Браузер поддерживает не только HTML, но также XML и SVG. Таким образом, класс `Element` служит основой для более специфичных классов.

6 И наконец, **HTMLElement** является базовым классом для всех остальных HTML-элементов. Мы будем работать с ним большую часть времени. От него наследуются элементы:

- `HTMLInputElement` - класс для тега `<input>`,
- `HTMLBodyElement` - класс для тега `<body>`,
- `HTMLAnchorElement` - класс для тега `<a>` и др.

Свойства узлов

Классы DOM-узлов



Таким образом, полный набор свойств и методов данного узла является результатом цепочки наследования.

Рассмотрим DOM-объект для тега `<input>`. Он принадлежит **HTMLInputElement**.

Он получает свойства и методы из (в порядке наследования):

- **HTMLInputElement** - этот класс предоставляет специфичные для элементов формы свойства,
- **HTMLElement** - предоставляет общие для HTML-элементов методы (и геттеры/сеттеры),
- **Element** - предоставляет типовые методы элемента,
- **Node** - предоставляет общие свойства DOM-узлов,
- **EventTarget** - обеспечивает поддержку событий (поговорим о них дальше),
- ...и, наконец, он наследует от **Object**, поэтому доступны также методы «обычного объекта», такие как `hasOwnProperty`.

Классы DOM-узлов

Для того, чтобы узнать имя класса DOM-узла, вспомним, что обычно у объекта есть свойство `constructor`. Оно ссылается на конструктор класса, и в свойстве `constructor.name` содержится его имя:

```
alert( document.body.constructor.name ); // HTMLBodyElement
```

Проверить наследование можно также при помощи `instanceof`:

```
alert( document.body instanceof HTMLBodyElement ); // true
alert( document.body instanceof HTMLElement ); // true
alert( document.body instanceof Element ); // true
alert( document.body instanceof Node ); // true
alert( document.body instanceof EventTarget ); // true
```

Классы DOM-узлов

Как видно, DOM-узлы - это обычные JavaScript объекты. Для наследования они используют классы, основанные на прототипах.

В этом легко убедиться, если вывести в консоли браузера любой элемент через `console.dir(elem)`. Или даже напрямую обратиться к методам, которые хранятся в `HTMLElement.prototype`, `Element.prototype` и т.д.

Большинство браузеров поддерживают в инструментах разработчика две команды: `console.log` и `console.dir`. Они выводят свои аргументы в консоль. Для JavaScript-объектов эти команды обычно выводят одно и то же. Но для DOM-элементов они работают по-разному:

- `console.log(elem)` выводит элемент в виде DOM-дерева.
- `console.dir(elem)` выводит элемент в виде DOM-объекта, что удобно для анализа его свойств.

Свойство `nodeType` предоставляет ещё один, «старомодный» способ узнать «тип» DOM-узла.

Его значением является цифра (пример представлен ниже):

- `elem.nodeType == 1` для узлов-элементов,
- `elem.nodeType == 3` для текстовых узлов,
- `elem.nodeType == 9` для объектов документа,
- В спецификации <https://dom.spec.whatwg.org/#node> можно посмотреть остальные значения.

В современных скриптах, чтобы узнать тип узла, мы можем использовать метод `instanceof` и другие способы проверить класс, но иногда `nodeType` проще использовать. Мы не можем изменить значение `nodeType`, только прочесть его.

Свойство `nodeType`

Свойства узлов

```
<body>
  <script>
    let elem = document.body;

    // давайте разберёмся: какой тип узла находится в elem?
    alert(elem.nodeType); // 1 => элемент

    // и его первый потомок...
    alert(elem.firstChild.nodeType); // 3 => текст

    // для объекта document значение типа -- 9
    alert( document.nodeType ); // 9
  </script>
</body>
```

Получив DOM-узел, мы можем узнать имя его тега из свойств `nodeName` и `tagName`, например:

```
alert( document.body.nodeName ); // BODY
alert( document.body.tagName ); // BODY
```

Есть ли какая-то разница между `tagName` и `nodeName`?

Да, она отражена в названиях свойств, но не очевидна.

- Свойство `tagName` есть только у элементов `Element`.
- Свойство `nodeName` определено для любых узлов `Node`:
 - для элементов оно равно `tagName`.
 - для остальных типов узлов (текст, комментариев и т.д.) оно содержит строку с типом узла.

Другими словами, свойство `tagName` есть только у узлов-элементов (поскольку они происходят от класса `Element`), а `nodeName` может что-то сказать о других типах узлов.

Тег: `nodeName` и `tagName`

Свойства узлов

innerHTML: содержимое элемента

Свойство `innerHTML` позволяет получить HTML-содержимое элемента в виде строки.

Мы также можем изменять его. Это один из самых мощных способов менять содержимое на странице. Пример показывает содержимое `document.body`, а затем полностью заменяет его:

```
<body>
  <p>Параграф</p>
  <div>DIV</div>

  <script>
    alert( document.body.innerHTML ); // читаем текущее содержимое
    document.body.innerHTML = 'Новый BODY!'; // заменяем содержимое
  </script>
</body>
```

innerHTML: содержимое элемента

Мы можем попробовать вставить некорректный HTML, браузер исправит наши ошибки:

Если innerHTML вставляет в документ тег `<script>` - он становится частью HTML, но не запускается.

Будьте внимательны: `innerHTML+=` осуществляет перезапись. Другими словами, `innerHTML+=` делает следующее:

- Старое содержимое удаляется.
- На его место становится новое значение `innerHTML` (с добавленной строкой).

```
<body>  
  
  <script>  
    document.body.innerHTML = '<b>тест'; // забыли закрыть тег  
    alert( document.body.innerHTML ); // <b>тест</b> (исправлено)  
  </script>  
  
</body>
```

```
elem.innerHTML += "...";  
// это более короткая запись для:  
elem.innerHTML = elem.innerHTML + "..."
```

Свойство `outerHTML` содержит HTML элемента целиком. Это как `innerHTML` плюс сам элемент.

Пример (1):

Будьте осторожны: в отличие от `innerHTML`, запись в `outerHTML` не изменяет элемент. Вместо этого элемент заменяется целиком во внешнем контексте.

Да, звучит странно, и это действительно необычно, поэтому здесь отмечаем эту особенность. Рассмотрим пример (2):

```
<div id="elem">Привет <b>Мир</b></div>
```

```
<script>
| alert(elem.outerHTML); // <div id="elem">Привет <b>Мир</b></div>
</script>
```

```
<div>Привет, мир!</div>
```

```
<script>
| let div = document.querySelector('div');
|
| // заменяем div.outerHTML на <p>...</p>
| div.outerHTML = '<p>Новый элемент</p>'; // (*)
|
| // Содержимое div осталось тем же!
| alert(div.outerHTML); // <div>Привет, мир!</div> (**)
</script>
```

outerHTML: HTML элемента целиком

Свойства узлов

В строке (*) мы заменили `div` на `<p>Новый элемент</p>`. Во внешнем документе мы видим новое содержимое вместо `<div>`. Но, как видно в строке (**), старая переменная `div` осталась прежней!

Это потому, что использование `outerHTML` не изменяет DOM-элемент, а удаляет его из внешнего контекста и вставляет вместо него новый HTML-код.

То есть, при `div.outerHTML=...` произошло:

- `div` был удалён из документа.
- Вместо него был вставлен другой HTML `<p>Новый элемент</p>`.
- В `div` осталось старое значение. Новый HTML не сохранён ни в какой переменной.

```
<div>Привет, мир!</div>

<script>
  let div = document.querySelector('div');

  // заменяем div.outerHTML на <p>...</p>
  div.outerHTML = '<p>Новый элемент</p>'; // (*)

  // Содержимое div осталось тем же!
  alert(div.outerHTML); // <div>Привет, мир!</div> (**)
</script>
```

outerHTML: HTML элемента целиком

Свойства узлов

Здесь легко сделать ошибку: заменить `div.outerHTML`, а потом продолжить работать с `div`, как будто там новое содержимое. Но это не так. Подобное верно для `innerHTML`, но не для `outerHTML`.

Мы можем писать в `elem.outerHTML`, но надо иметь в виду, что это не меняет элемент, в который мы пишем. Вместо этого создаётся новый HTML на его месте. Мы можем получить ссылки на новые элементы, обратившись к DOM.

```
<div>Привет, мир!</div>

<script>
  let div = document.querySelector('div');

  // заменяем div.outerHTML на <p>...</p>
  div.outerHTML = '<p>Новый элемент</p>'; // (*)

  // Содержимое div осталось тем же!
  alert(div.outerHTML); // <div>Привет, мир!</div> (**)
</script>
```

outerHTML: HTML элемента целиком

Свойства узлов

nodeValue/data: содержимое текстового узла

Свойство `innerHTML` есть только у узлов-элементов.

У других типов узлов, в частности, у текстовых, есть свои аналоги: свойства `nodeValue` и `data`. Эти свойства очень похожи при использовании, есть лишь небольшие различия в спецификации. Мы будем использовать `data`, потому что оно короче.

Прочитаем содержимое текстового узла и комментария:

Мы можем представить, для чего нам может понадобиться читать или изменять текстовый узел, но комментарии? Иногда их используют для вставки информации и инструкций шаблонизатора в HTML, как в примере

```
<!-- if isAdmin -->
| <div>Добро пожаловать, Admin!</div>
<!-- /if -->
```

```
<body>
Привет
<!-- Комментарий -->
<script>
  let text = document.body.firstChild;
  alert(text.data); // Привет

  let comment = text.nextSibling;
  alert(comment.data); // Комментарий
</script>
</body>
```

Свойства узлов

textContent: просто текст

Свойство textContent предоставляет доступ к тексту внутри элемента за вычетом всех <тегов>. Например:

```
<div id="news">
  <h1>Срочно в номер!</h1>
  <p>Марсиане атаковали человечество!</p>
</div>

<script>
  // Срочно в номер! Марсиане атаковали человечество!
  alert(news.textContent);
</script>
```

Как мы видим, возвращается только текст, как если бы все <теги> были вырезаны, но текст в них остался.

На практике редко появляется необходимость читать текст таким образом.

textContent: просто текст

Намного полезнее возможность записывать текст в `textContent`, т.к. позволяет писать текст «безопасным способом». Представим, что у нас есть произвольная строка, введенная пользователем, и мы хотим показать её.

- С `innerHTML` вставка происходит «как HTML», со всеми HTML-тегами.
- С `textContent` вставка получается «как текст», все символы трактуются буквально.

Сравним два тега `div`:

```
<div id="elem1"></div>
<div id="elem2"></div>

<script>
  let name = prompt("Введите ваше имя?", "<b>Человек-паук!</b>");

  elem1.innerHTML = name;
  elem2.textContent = name;
</script>
```

textContent: просто текст

- В первый <div> имя приходит «как HTML»: все теги стали именно тегами, поэтому мы видим имя, выделенное жирным шрифтом.
- Во второй <div> имя приходит «как текст», поэтому мы видим Человек-паук!.

В большинстве случаев мы рассчитываем получить от пользователя текст и хотим, чтобы он интерпретировался как текст. Мы не хотим, чтобы на сайте появлялся произвольный HTML-код.

Присваивание через
textContent - один
из способов от этого
защититься.

```
<div id="elem1"></div>
<div id="elem2"></div>

<script>
  let name = prompt("Введите ваше имя?", "<b>Человек-паук!</b>");

  elem1.innerHTML = name;
  elem2.textContent = name;
</script>
```

Атрибут и DOM-свойство `hidden` указывает на то, видим ли мы элемент или нет.

Мы можем использовать его в HTML или назначать при помощи JavaScript, как в примере ^

Технически, `hidden` работает так же, как `style="display:none"`. Но его применение проще.

Мигающий элемент ->

```
<div>Оба тега DIV внизу невидимы</div>

<div hidden>С атрибутом "hidden"</div>

<div id="elem">С назначенным **JavaScript** свойством "hidden"</div>

<script>
|   elem.hidden = true;
</script>
```

```
<div id="elem">Мигающий элемент</div>

<script>
|   setInterval(() => elem.hidden = !elem.hidden, 1000);
</script>
```

Свойство `hidden`

Свойства узлов

У DOM-элементов есть дополнительные свойства, в частности, зависящие от класса:

- `value` - значение для `<input>`, `<select>` и `<textarea>` (`HTMLInputElement`, `HTMLSelectElement...`).
- `href` - адрес ссылки `"href"` для `<a href="...">` (`HTMLAnchorElement`).
- `id` - значение атрибута `"id"` для всех элементов (`HTMLElement`) и другие.

Большинство стандартных HTML-атрибутов имеют соответствующее DOM-свойство, и мы можем получить к нему доступ. Если хотим узнать полный список поддерживаемых свойств для данного класса, можно найти их в спецификации. Например, класс `HTMLInputElement` описывается **здесь** <https://html.spec.whatwg.org/#htmlinputelement> .

Если нас интересует специфика определённого браузера - можем вывести элемент в консоль, используя `console.dir(elem)`, и прочитать все свойства. Или исследовать свойства DOM во вкладке `Elements` браузерных инструментов разработчика.

Другие свойства

Свойства узлов

```
<input type="text" id="elem" value="значение">
```

```
<script>  
  alert(elem.type); // "text"  
  alert(elem.id);  // "elem"  
  alert(elem.value); // значение  
</script>
```

6

ИЗМЕНЕНИЕ ДОКУМЕНТА

Модификации DOM - это ключ к созданию «живых» страниц. Рассмотрим, как создавать новые элементы «на лету» и изменять уже существующие.

Рассмотрим методы на примере - а именно, добавим на страницу сообщение, которое будет выглядеть получше, чем alert.

Это был пример HTML. Давайте создадим такой же div, используя JavaScript

```
<style>
.alert {
  padding: 15px;
  border: 1px solid #d6e9c6;
  border-radius: 4px;
  color: #3c763d;
  background-color: #dfff0d8;
}
</style>

<div class="alert">
  <strong>Всем привет!</strong> Вы прочитали важное сообщение.
</div>
```

Создание элемента

Изменение документа

DOM-узел можно создать двумя методами:

- `document.createElement(tag)` : Создаёт новый элемент с заданным тегом

```
let div = document.createElement('div');
```

- `document.createTextNode(text)` : Создаёт
новый текстовый узел с заданным текстом

```
let textNode = document.createTextNode('А вот и я');
```

Создание элемента

Изменение документа

В нашем случае сообщение - это div с классом alert и HTML в нём:

```
let div = document.createElement('div');  
div.className = "alert";  
div.innerHTML = "<strong>Всем привет!</strong> Вы прочитали важное сообщение.";
```

Мы создали элемент, но пока он только в переменной. Мы не можем видеть его на странице, поскольку он не является частью документа.

Чтобы наш div появился, нам нужно вставить его где-нибудь в document.
Например, в document.body.

Для этого есть метод append, в нашем случае:

```
document.body.append(div);
```

Создание сообщения и методы вставки

Изменение документа

Вот методы для различных вариантов вставки:

- `node.append(...nodes or strings)` - добавляет узлы или строки в конец `node`,
- `node.prepend(...nodes or strings)` - вставляет узлы или строки в начало `node`,
- `node.before(...nodes or strings)` - вставляет узлы или строки до `node`,
- `node.after(...nodes or strings)` - вставляет узлы или строки после `node`,
- `node.replaceWith(...nodes or strings)` - заменяет `node` заданными узлами или строками.

Вот пример использования этих методов, чтобы добавить новые элементы в список и текст до/после него:

Методы вставки

Изменение документа

```
<ol id="ol">
  <li>0</li>
  <li>1</li>
  <li>2</li>
</ol>

<script>
  ol.before('before'); // вставить строку "before" перед <ol>
  ol.after('after'); // вставить строку "after" после <ol>

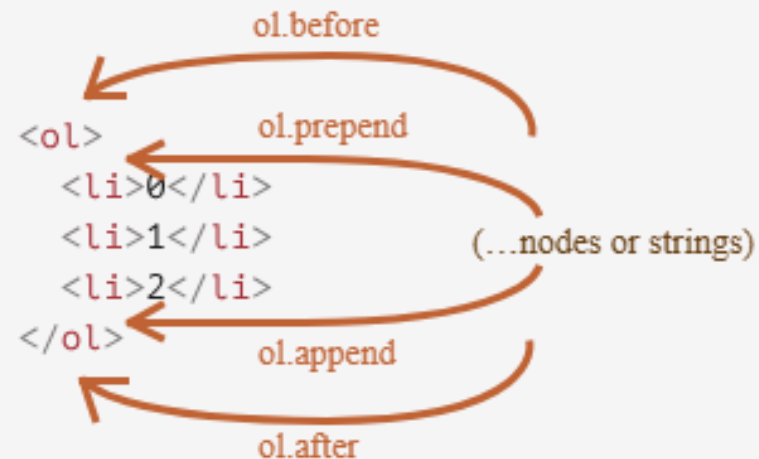
  let liFirst = document.createElement('li');
  liFirst.innerHTML = 'prepend';
  ol.prepend(liFirst); // вставить liFirst в начало <ol>

  let liLast = document.createElement('li');
  liLast.innerHTML = 'append';
  ol.append(liLast); // вставить liLast в конец <ol>
</script>
```

Наглядная иллюстрация того, куда эти методы вставляют:

Итоговый список будет таким:

```
before
<ol id="ol">
  <li>prepend</li>
  <li>0</li>
  <li>1</li>
  <li>2</li>
  <li>append</li>
</ol>
after
```



```
<ol id="ol">
  <li>0</li>
  <li>1</li>
  <li>2</li>
</ol>

<script>
  ol.before('before'); // вставить строку "before" перед <ol>
  ol.after('after'); // вставить строку "after" после <ol>

  let liFirst = document.createElement('li');
  liFirst.innerHTML = 'prepend';
  ol.prepend(liFirst); // вставить liFirst в начало <ol>

  let liLast = document.createElement('li');
  liLast.innerHTML = 'append';
  ol.append(liLast); // вставить liLast в конец <ol>
</script>
```

Методы вставки

Изменение документа

Если хотим вставить HTML именно "как html"

С этим может помочь другой, довольно универсальный метод:
`elem.insertAdjacentHTML(where, html).`

Первый параметр - это специальное слово, указывающее, куда по отношению к `elem` производить вставку. Значение должно быть одним из следующих:

- `"beforebegin"` -- вставить `html` непосредственно перед `elem`,
- `"afterbegin"` -- вставить `html` в начало `elem`,
- `"beforeend"` -- вставить `html` в конец `elem`,
- `"afterend"` -- вставить `html` непосредственно после `elem`.

Второй параметр - это HTML-строка, которая будет вставлена именно "как HTML".

Изменение документа

Если хотим вставить HTML именно "как html"

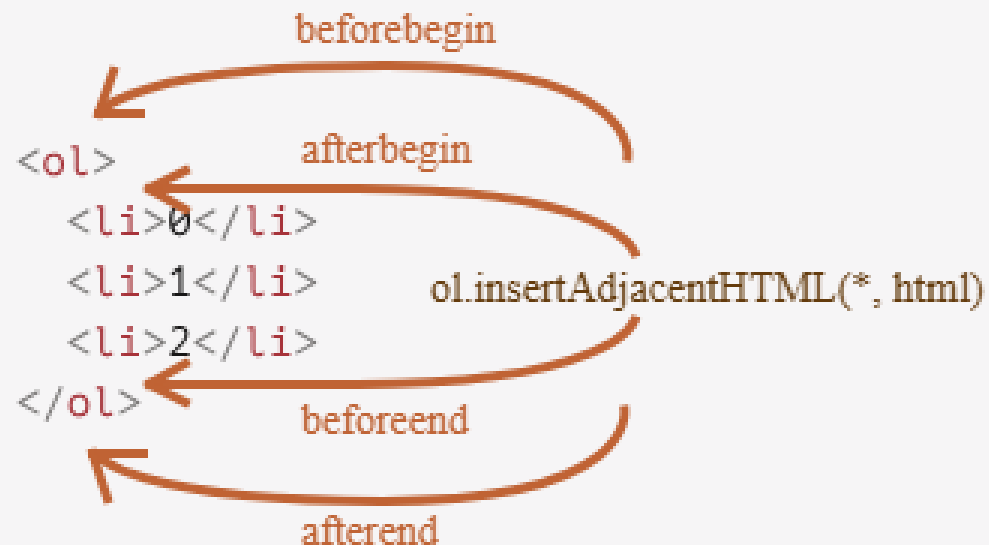
```
<div id="div"></div>
<script>
  div.insertAdjacentHTML('beforebegin', '<p>Привет</p>');
  div.insertAdjacentHTML('afterend', '<p>Пока</p>');
</script>
```

```
<p>Привет</p>
<div id="div"></div>
<p>Пока</p>
```

Так, код ^, приведет к ->

Так мы можем добавлять произвольный HTML на страницу.

Варианты вставки представлены ->



Изменение документа

Если хотим вставить HTML именно "как html"

Есть ещё один, очень древний метод добавления содержимого на веб-страницу: `document.write`.

```
<p>Где-то на странице...</p>
<script>
|   document.write('<b>Привет из JS</b>');
</script>
<p>Конец</p>
```

Вызов `document.write(html)` записывает html на страницу «прямо здесь и сейчас». Строка html может быть динамически сгенерирована, поэтому метод достаточно гибкий. Мы можем использовать JavaScript, чтобы создать полноценную веб-страницу и записать её в документ.

В современных скриптах он редко встречается из-за следующего важного ограничения:

Вызов `document.write` работает только во время загрузки страницы.

7

СТИЛИ И КЛАССЫ

JavaScript и стили

Как правило, существует два способа задания стилей для элемента:

- Создать класс в CSS и использовать его: `<div class="...">`
- Писать стили непосредственно в атрибуте style: `<div style="...">`.

JavaScript может менять и классы, и свойство style.

Классы - всегда предпочтительный вариант по сравнению со style. Мы должны манипулировать свойством style только в том случае, если классы «не могут справиться».

Например, использование style является приемлемым, если мы вычисляем координаты элемента динамически и хотим установить их из JavaScript:

```
let top = /* сложные расчёты */;  
let left = /* сложные расчёты */;  
  
elem.style.left = left; // например, '123px',  
// значение вычисляется во время работы скрипта  
elem.style.top = top; // например, '456px'
```

Изменение класса является одним из наиболее часто используемых действий в скриптах.

Когда-то давно в JavaScript существовало ограничение: зарезервированное слово типа "class" не могло быть свойством объекта. Это ограничение сейчас отсутствует, но в то время было невозможно иметь свойство `elem.class`.

Поэтому для классов было введено схожее свойство "className": `elem.className` соответствует атрибуту "class".

```
<body class="main page">  
  <script>  
    alert(document.body.className); // main page  
  </script>  
</body>
```

Если мы присваиваем что-то `elem.className`, то это заменяет всю строку с классами. Иногда это то, что нам нужно, но часто мы хотим добавить/удалить один класс. Для этого есть другое свойство: `elem.classList`.

className и classList

`elem.classList` - это специальный объект с методами для добавления/удаления одного класса.

Так что мы можем работать как со строкой полного класса, используя `className`, так и с отдельными классами, используя `classList`. Выбираем тот вариант, который нам удобнее.

Методы `classList`:

- `elem.classList.add/remove("class")` - добавить/удалить класс.
- `elem.classList.toggle("class")` - добавить класс, если его нет, иначе удалить.
- `elem.classList.contains("class")` - проверка наличия класса, возвращает `true/false`.

Кроме того, `classList` является перебираемым, поэтому можно перечислить все классы при помощи `for..of`

```
<body class="main page">
  <script>
    for (let name of document.body.classList) {
      alert(name); // main, затем page
    }
  </script>
</body>
```

className и classList

Свойство style

Свойство `elem.style` - это объект, который соответствует тому, что написано в атрибуте "style".

Свойства из одного слова
записываются так же, с маленькой буквы:

```
background => elem.style.background
top        => elem.style.top
opacity    => elem.style.opacity
```

Для свойств из нескольких слов
используется camelCase:

```
background-color => elem.style.backgroundColor
z-index          => elem.style.zIndex
border-left-width => elem.style.borderLeftWidth
```

Стили с браузерным префиксом, например, `-moz-border-radius`, `-webkit-border-radius` преобразуются по тому же принципу: дефис означает заглавную букву.

```
button.style.MozBorderRadius = '5px';
button.style.WebkitBorderRadius = '5px';
```

Сброс стилей

Иногда нам нужно добавить свойство стиля, а потом, позже, убрать его.

Например, чтобы скрыть элемент, мы можем задать `elem.style.display = "none"`.

Затем мы можем удалить свойство `style.display`, чтобы вернуться к первоначальному состоянию. Вместо `delete elem.style.display` мы должны присвоить ему пустую строку: `elem.style.display = ""`.

```
// если мы запустим этот код, <body> "мигнёт"  
document.body.style.display = "none"; // скрыть  
  
setTimeout(() => document.body.style.display = "", 1000); // возврат к нормальному состоянию
```

Если мы установим в `style.display` пустую строку, то браузер применит CSS-классы и встроенные стили, как если бы такого свойства `style.display` вообще не было.

8

БРАУЗЕРНЫЕ СОБЫТИЯ

Основные браузерные события

Событие - это сигнал от браузера о том, что что-то произошло. Все DOM-узлы подают такие сигналы (хотя события бывают и не только в DOM). Самые часто используемые события:

События мыши:

- `click` – происходит, когда кликнули на элемент левой кнопкой мыши (на устройствах с сенсорными экранами оно происходит при касании).
- `contextmenu` – происходит, когда кликнули на элемент правой кнопкой мыши.
- `mouseover` / `mouseout` – когда мышь наводится на / покидает элемент.
- `mousedown` / `mouseup` – когда нажали / отжали кнопку мыши на элементе.
- `mousemove` – при движении мыши.

Основные браузерные события

События на элементах управления:

- `submit` – пользователь отправил форму `<form>`.
- `focus` – пользователь фокусируется на элементе, например нажимает на `<input>`.

Клавиатурные события:

- `keydown` и `keyup` – когда пользователь нажимает / отпускает клавишу.

События документа:

- `DOMContentLoaded` – когда HTML загружен и обработан, DOM документа полностью построен и доступен.

CSS events:

- `transitionend` – когда CSS-анимация завершена.

Событию можно назначить обработчик, то есть функцию, которая сработает, как только событие произошло.

Именно благодаря обработчикам JavaScript-код может реагировать на действия пользователя.

Есть несколько способов назначить событию обработчик. Обработчик может быть назначен прямо в разметке, в атрибуте, который называется on<событие>.

Например, чтобы назначить обработчик события click на элементе input, можно использовать атрибут onclick, вот так:

```
<input value="Нажми меня" onclick="alert('Клик!')" type="button">
```

При клике мышкой на кнопке выполнится код, указанный в атрибуте onclick. Для содержимого атрибута onclick используются одинарные кавычки, так как сам атрибут находится в двойных. Если поставим двойные кавычки внутри атрибута: onclick="alert("Click!")", код не будет работать.

Обработчик события

Стили и классы

Атрибут HTML-тега - не самое удобное место для написания большого количества кода, поэтому лучше создать отдельную JavaScript-функцию и вызвать её там.

Следующий пример
по клику запускает
функцию countRabbits():

```
<script>
  function countRabbits() {
    for(let i=1; i<=3; i++) {
      alert("Кролик номер " + i);
    }
  }
</script>

<input type="button" onclick="countRabbits()" value="Считать кроликов!">
```

Обработчик события

Стили и классы

Можно назначать обработчик, используя свойство DOM-элемента `on<событие>`.
Например, `elem.onclick`

Если обработчик задан через атрибут, то браузер читает HTML-разметку, создаёт новую функцию из содержимого атрибута и записывает в свойство.

```
<input id="elem" type="button" value="Нажми меня!">
<script>
  elem.onclick = function() {
    alert('Спасибо');
  };
</script>
```

Этот способ, по сути, аналогичен предыдущему.

Обработчик всегда хранится в свойстве DOM-объекта, а атрибут – лишь один из способов его инициализации. Так как у элемента DOM может быть только одно свойство с именем `onclick`, то назначить более одного обработчика так нельзя.

Обработчиком можно назначить и уже существующую функцию.

Обработчик события

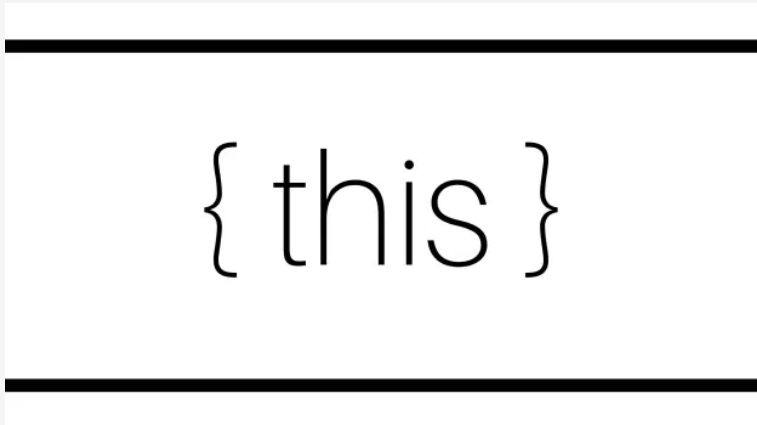
Стили и классы

Доступ к элементу через this

Внутри обработчика события `this` ссылается на текущий элемент, то есть на тот, на котором, как говорят, «висит» (т.е. назначен) обработчик.

В коде ниже `button` выводит своё содержимое, используя `this.innerHTML`

```
<button onclick="alert(this.innerHTML)">Нажми меня</button>
```



The diagram shows a white rectangular box with a thin black border. Inside the box, the text `{ this }` is centered. The box is flanked by vertical grey bars on the left and right sides.

```
{ this }
```

Браузерные события

addEventListener

Фундаментальный недостаток описанных выше способов назначения обработчика – невозможность повесить несколько обработчиков на одно событие. Например, одна часть кода хочет при клике на кнопку делать её подсвеченной, а другая – выдавать сообщение.

Мы хотим назначить два обработчика для этого. Но новое DOM-свойство перезапишет предыдущее:

```
input.onclick = function() { alert(1); }  
// ...  
input.onclick = function() { alert(2); } // заменит предыдущий обработчик
```

Разработчики стандартов достаточно давно это поняли и предложили альтернативный способ назначения обработчиков при помощи специальных методов `addEventListener` и `removeEventListener`. Они свободны от указанного недостатка.

Браузерные события

addEventListener

Синтаксис добавления обработчика:

```
element.addEventListener(event, handler, [options]);
```

- event : Имя события, например "click".
- handler : Ссылка на функцию-обработчик.
- options : Дополнительный объект со свойствами: - once: если true, тогда обработчик будет автоматически удалён после выполнения. - capture: фаза, на которой должен сработать обработчик. Так исторически сложилось, что options может быть false/true, это то же самое, что {capture: false/true}. - passive: если true, то указывает, что обработчик никогда не вызовет preventDefault().

Для удаления обработчика следует использовать removeEventListener:

```
element.removeEventListener(event, handler, [options]);
```

Для удаления нужно передать именно ту функцию-обработчик которая была назначена.

Браузерные события

addEventListener

Обратим внимание - если функцию обработчик не сохранить где-либо, мы не сможем её удалить. Нет метода, который позволяет получить из элемента обработчики событий, назначенные через `addEventListener`.

Метод `addEventListener` позволяет добавлять несколько обработчиков на одно событие одного элемента, например:

```
<input id="elem" type="button" value="Нажми меня"/>

<script>
  function handler1() {
    alert('Спасибо!');
  };

  function handler2() {
    alert('Спасибо ещё раз!');
  }

  elem.onclick = () => alert("Привет");
  elem.addEventListener("click", handler1); // Спасибо!
  elem.addEventListener("click", handler2); // Спасибо ещё раз!
</script>
```

9

ВСПЛЫТИЕ И ПОГРУЖЕНИЕ

Всплытие и погружение

Всплытие

Принцип всплытия прост: когда на элементе происходит событие, обработчики сначала срабатывают на нём, потом на его родителе, затем выше и так далее, вверх по цепочке предков.

Например, есть 3 вложенных элемента
FORM > DIV > P с обработчиком на каждом

Клик по внутреннему <p> вызовет обработчик onclick:

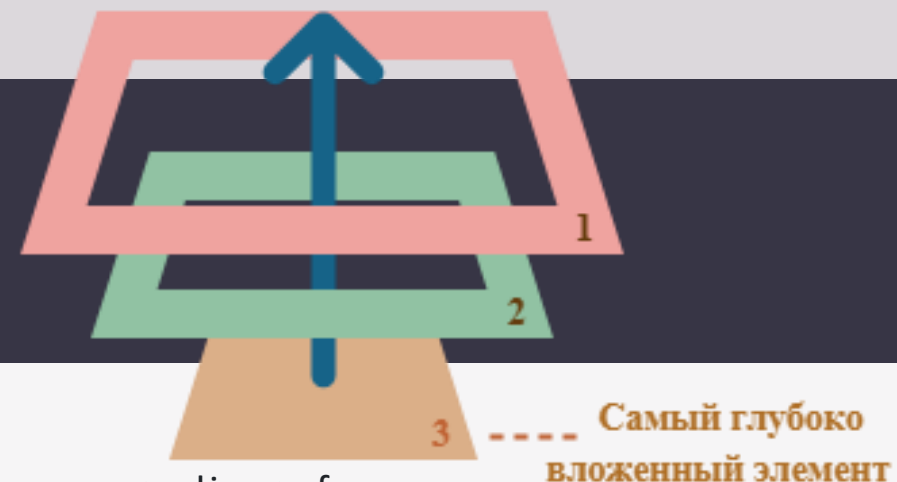
- Сначала на самом <p>.
- Потом на внешнем <div>.
- Затем на внешнем <form>.
- И так далее вверх по цепочке до самого document.

```
<style>
  body * {
    margin: 10px;
    border: 1px solid blue;
  }
</style>

<form onclick="alert('form')">FORM
  <div onclick="alert('div')">DIV
    <p onclick="alert('p')">P</p>
  </div>
</form>
```

Всплытие и погружение

Всплытие



Поэтому если кликнуть на `<p>`, то мы увидим три оповещения: `p -> div -> form`.

Этот процесс называется «всплытием», потому что события «всплывают» от внутреннего элемента вверх через родителей.

Есть и исключения.

Например, событие `focus` не всплывает. В дальнейшем мы увидим и другие примеры. Однако, стоит понимать, что это скорее исключение, чем правило, всё-таки большинство событий всплывают.

```
<style>
  body * {
    margin: 10px;
    border: 1px solid blue;
  }
</style>

<form onclick="alert('form')">FORM
  <div onclick="alert('div')">DIV
    <p onclick="alert('p')">P</p>
  </div>
</form>
```

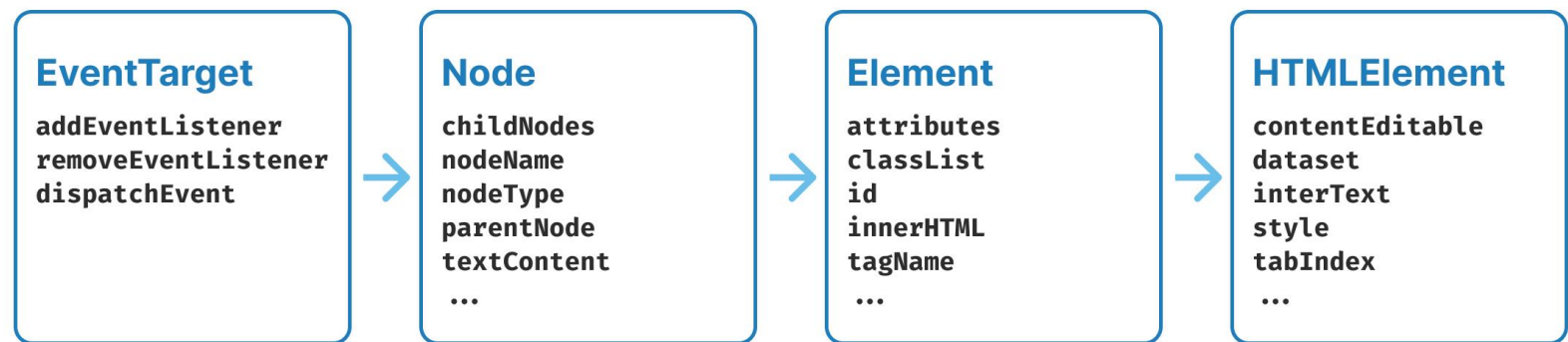
Всегда можно узнать, на каком конкретно элементе произошло событие.

Самый глубокий элемент, который вызывает событие, называется целевым элементом, и он доступен через `event.target`.

Отличия от `this` (`=event.currentTarget`):

- `event.target` - это «целевой» элемент, на котором произошло событие, в процессе всплытия он неизменен.
- `this` - это «текущий» элемент, до которого дошло всплытие, на нём сейчас выполняется обработчик.

event.target



Например, если стоит только один обработчик `form.onclick`, то он «поймает» все клики внутри формы. Где бы ни был клик внутри -- он всплывёт до элемента `<form>`, на котором сработает обработчик.

При этом внутри обработчика `form.onclick`:

- `this` (`=event.currentTarget`) всегда будет элемент `<form>`, так как обработчик сработал на ней.
- `event.target` будет содержать ссылку на конкретный элемент внутри формы, на котором произошёл клик.

Возможна и ситуация, когда `event.target` и `this` - один и тот же элемент, например, если клик был непосредственно на самом элементе `<form>`, а не на его подэлементе.

event.target

Всплытие и погружение

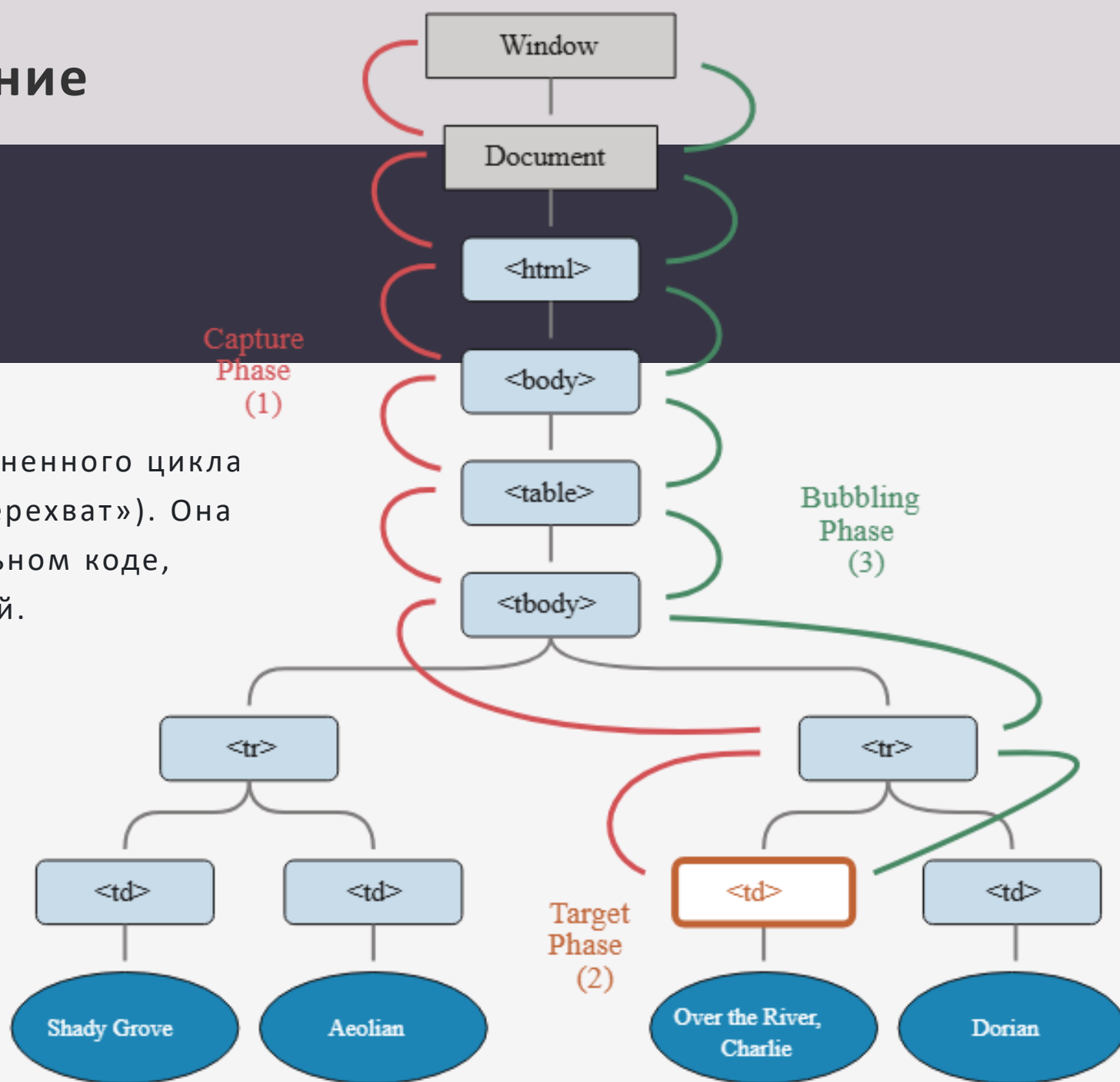
Всплытие и погружение

Погружение

Существует ещё одна фаза из жизненного цикла события – «погружение» (или «перехват»). Она очень редко используется в реальном коде, однако тоже может быть полезной.

Стандарт DOM Events описывает 3 фазы прохода события:

- Фаза погружения - событие сначала идёт сверху вниз.
- Фаза цели - событие достигло целевого(исходного) элемента.
- Фаза всплытия – событие начинает всплывать.



Всплытие и погружение

Делегирование

Всплытие и перехват событий позволяет реализовать один из самых важных приёмов разработки - делегирование.

Идея в том, что если у нас есть много элементов, события на которых нужно обрабатывать похожим образом, то вместо того, чтобы назначать обработчик каждому, мы ставим один обработчик на их общего предка.

Из него можно получить целевой элемент `event.target`, понять на каком именно потомке произошло событие и обработать его.



Применение делегирования: действия в разметке

Например, нам нужно сделать меню с разными кнопками: «Сохранить (save)», «Загрузить (load)», «Поиск (search)» и т.д. И есть объект с соответствующими методами `save`, `load`, `search`... Как их состыковать?

Первое, что может прийти в голову – это найти каждую кнопку и назначить ей свой обработчик среди методов объекта. Но существует более элегантное решение.

Мы можем добавить один обработчик для всего меню и атрибуты `data-action` для каждой кнопки в соответствии с методами, которые они вызывают:

```
<button data-action="save">Нажмите, чтобы Сохранить</button>
```

Обработчик считывает содержимое атрибута и выполняет метод.

Всплытие и погружение

Пример

Метод `this.onClick` в строке (*), привязывается к контексту текущего объекта `this`. Это важно, т.к. иначе `this` внутри него будет ссылаться на DOM-элемент (`elem`), а не на объект `Menu`, и `this[action]` будет не тем, что нам нужно.

Так что же даёт нам здесь делегирование? Не нужно писать код, чтобы присвоить обработчик каждой кнопке. Достаточно просто создать один метод и поместить его в разметку. Структура HTML становится по-настоящему гибкой. Мы можем добавлять/удалять кнопки в любое время. Можем использовать классы `.action-save`, `.action-load`, но подход с использованием атрибутов `data-action` является более семантическим.

```
<div id="menu">
  <button data-action="save">Сохранить</button>
  <button data-action="load">Загрузить</button>
  <button data-action="search">Поиск</button>
</div>

<script>
  class Menu {
    constructor(elem) {
      | elem.onclick = this.onClick.bind(this); // (*)
    }

    save() {
      | alert('сохраняю');
    }

    load() {
      | alert('загружаю');
    }

    search() {
      | alert('ищу');
    }

    onClick(event) {
      | let action = event.target.dataset.action;
      | if (action) {
      |   this[action]();
      | }
    }
  }

  new Menu(menu);
</script>
```

Делегирование событий можно использовать для добавления элементам «поведения» (behavior), декларативно задавая хитрые обработчики установкой специальных HTML-атрибутов и классов.

Приём проектирования «поведение» состоит из двух частей:

- Элементу ставится пользовательский атрибут, описывающий его поведение.
- При помощи делегирования ставится обработчик на документ, который ловит все клики (или другие события) и, если элемент имеет нужный атрибут, производит соответствующее действие.

behavior

Всплытие и погружение

Например, здесь HTML-атрибут data-counter добавляет кнопкам поведение: «увеличить значение при клике»

Если нажать на кнопку - значение увеличится. Конечно, нам важны не счётчики, а общий подход, который здесь продемонстрирован.

Элементов с атрибутом data-counter может быть сколько угодно.

Новые могут добавляться в HTML-код в любой момент. При помощи делегирования мы фактически добавили новый «псевдостандартный» атрибут в HTML, который добавляет элементу новую возможность («поведение»).

```
Счётчик: <input type="button" value="1" data-counter>
Ещё счётчик: <input type="button" value="2" data-counter>

<script>
  document.addEventListener('click', function(event) {
    if (event.target.dataset.counter != undefined) { // если есть атрибут...
      event.target.value++;
    }
  });
</script>
```

Поведение: "Счётчик"

Всплытие и погружение

Ещё один пример поведения. Сделаем так, что при клике на элемент с атрибутом data-toggle-id будет скрываться/показываться элемент с заданным id

Ещё раз подчеркнём, что мы сделали. Теперь для того, чтобы добавить скрытие-раскрытие любому элементу, даже не надо знать JavaScript, можно просто написать атрибут data-toggle-id.

Это бывает очень удобно - не нужно писать JavaScript-код для каждого элемента, который должен так себя вести.

Просто используем поведение. Обработчики на уровне документа сделают это возможным для элемента в любом месте страницы.

```
<button data-toggle-id="subscribe-mail">
  Показать форму подписки
</button>

<form id="subscribe-mail" hidden>
  Ваша почта: <input type="email">
</form>

<script>
  document.addEventListener('click', function(event) {
    let id = event.target.dataset.toggleId;
    if (!id) return;

    let elem = document.getElementById(id);

    elem.hidden = !elem.hidden;
  });
</script>
```

Поведение: "Переключатель" (Toggler)

Всплытие и погружение

10

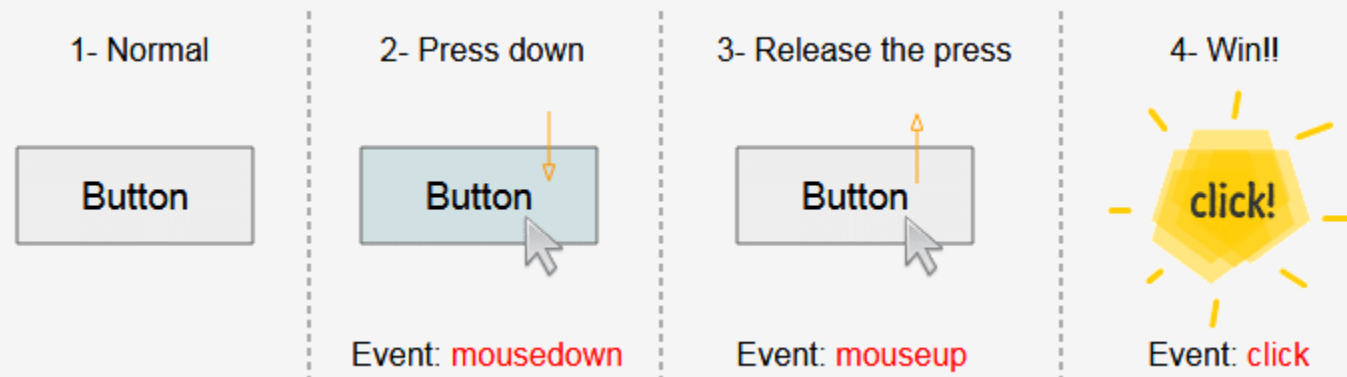
ГЕНЕРАЦІЯ СОБЫТІЙ

Можно не только назначать обработчики, но и генерировать события из JavaScript-кода.

Пользовательские события могут быть использованы при создании графических компонентов. Например, корневой элемент нашего меню, реализованного при помощи JavaScript, может генерировать события, относящиеся к этому меню: `open` (меню раскрыто), `select` (выбран пункт меню) и т.п. А другой код может слушать эти события и узнавать, что происходит с меню.

Можно генерировать не только совершенно новые, придуманные нами события, но и встроенные, такие как `click`, `mousedown` и другие.

Это бывает полезно для автоматического тестирования.



События

Генерация событий

Встроенные классы для событий формируют иерархию аналогично классам для DOM-элементов. Её корнем является встроенный класс Event. Событие встроенного класса Event можно создать так,

где:

```
let event = new Event(type[, options]);
```

- type - тип события, строка, например "click" или же любой придуманный нами - "my-event".
- options - объект с тремя необязательными свойствами:
 - bubbles: true/false - если true, тогда событие всплывает.
 - cancelable: true/false - если true, тогда можно отменить действие по умолчанию. Позже мы разберём, что это значит для пользовательских событий.
 - composed: true/false - если true, тогда событие будет всплывать наружу за пределы Shadow DOM.

По умолчанию все три свойства false: {bubbles: false, cancelable: false, composed: false}.

Конструктор Event

Генерация событий

Метод `dispatchEvent`

После того, как объект события создан, мы должны запустить его на элементе, вызвав метод `elem.dispatchEvent(event)`.

Затем обработчики отреагируют на него, как будто это обычное браузерное событие. Если при создании указан флаг `bubbles`, то оно будет всплывать.

В примере ниже событие `click` иницируется JavaScript-кодом так, как будто кликнули по кнопке:

Автоклик

```
<script> let event = new Event("click"); elem.dispatchEvent(event); </script>
```

Можно легко отличить «настоящее» событие от сгенерированного кодом.

Свойство `event.isTrusted` принимает значение `true` для событий, порождаемых реальными действиями пользователя, и `false` для генерируемых кодом.

Генерация событий

MouseEvent, KeyboardEvent и другие

Для некоторых конкретных типов событий есть свои специфические конструкторы. Вот небольшой список конструкторов для различных событий пользовательского интерфейса, которые можно найти в спецификации **UI Event** <https://www.w3.org/TR/uievents/> :

Стоит использовать их вместо `new Event`, если мы хотим создавать такие события.

К примеру, `new MouseEvent("click")`.

Mouse Events	Keyboard Events	Form Events	Document/Window Events
click	keypress	submit	load
dblclick	keydown	change	resize
mouseenter	keyup	focus	scroll
mouseleave		blur	unload

MouseEvent, KeyboardEvent и другие

Специфический конструктор позволяет указать стандартные свойства для данного типа события. Например, `clientX/clientY` для события мыши:

Обратите внимание: этого нельзя было бы сделать с обычным конструктором `Event`.

Впрочем, использование конкретного конструктора не является обязательным, можно обойтись `Event`, а свойства записать в объект отдельно, после создания, вот так: `event.clientX=100`. Здесь это скорее вопрос удобства и желания следовать правилам. События, которые генерирует браузер, всегда имеют правильный тип.

Полный список свойств по типам событий вы найдёте в спецификации.

```
let event = new MouseEvent("click", {  
  bubbles: true,  
  cancelable: true,  
  clientX: 100,  
  clientY: 100  
});  
  
alert(event.clientX); // 100
```

Пользовательские события

Для генерации событий совершенно новых типов, таких как "hello", следует использовать конструктор `new CustomEvent`. Технически `CustomEvent` абсолютно идентичен `Event` за исключением одной небольшой детали.

У второго аргумента-объекта есть дополнительное свойство `detail`, в котором можно указывать информацию для передачи в событие.

`detail` может содержать любые данные. Никто не мешает и в обычное `new Event` записать любые свойства. Но `CustomEvent`

предоставляет специальное поле `detail` во избежание конфликтов с другими свойствами события. Кроме того, класс события описывает, что это за событие, и если оно не браузерное, а пользовательское, то лучше использовать `CustomEvent`, чтобы явно об этом сказать.

```
<h1 id="elem">Привет для Васи!</h1>
```

```
<script>
```

```
  // дополнительная информация приходит в обработчик вместе с событием
  elem.addEventListener("hello", function(event) {
    alert(*!event.detail.name*!*);
  });
```

```
  elem.dispatchEvent(new CustomEvent("hello", {
    detail: { name: "Вася" }
  }));
```

```
</script>
```

`event.preventDefault()`

Для многих браузерных событий есть «действия по умолчанию», такие как переход по ссылке, выделение и т.п.

Для новых, пользовательских событий браузерных действий, конечно, нет, но код, который генерирует такое событие, может предусматривать какие-то свои действия после события.

Вызов `event.preventDefault()` является возможностью для обработчика события сообщить в сгенерировавший событие код, что эти действия надо отменить.

Тогда вызов `elem.dispatchEvent(event)` возвратит `false`. И код, сгенерировавший событие, узнает, что продолжать не нужно.

Генерация событий

event.preventDefault()

Посмотрим практический пример - прячущегося кролика. Можете видеть кролика #rabbit и функцию hide(), которая при вызове генерирует на нём событие "hide", уведомляя всех интересующихся, что кролик собирается спрятаться.

Сообщение с 127.0.0.1:5500:

Вызвать preventDefault?

ОК

Отмена

```
<pre id="rabbit">
```

```
| \    / |  
| \_  / |  
| / . . \ |  
| =\_Y\_ / =  
| {>o< }  
</pre>
```

```
</pre>
```

```
<button onclick="hide()">Hide()</button>
```

```
<script>
```

```
// hide() будет вызван при щелчке на кнопке
```

```
function hide() {
```

```
  let event = new CustomEvent("hide", {
```

```
    cancelable: true // без этого флага preventDefault не работает
```

```
  });
```

```
  if (!rabbit.dispatchEvent(event)) {
```

```
    alert('Действие отменено обработчиком');
```

```
  } else {
```

```
    rabbit.hidden = true;
```

```
  }
```

```
}
```

```
rabbit.addEventListener('hide', function(event) {
```

```
  if (confirm("Вызвать preventDefault?")) {
```

```
    event.preventDefault();
```

```
  }
```

```
});
```

```
</script>
```

```
| \    / |  
| \_  / |  
| / . . \ |  
| =\_Y\_ / =  
| {>o< }  
Hide()
```

Hide()

Генерация событий

event.preventDefault()

Любой обработчик может узнать об этом, подписавшись на событие hide через `rabbit.addEventListener('hide',...)` и, при желании, отменить действие по умолчанию через `event.preventDefault()`. Тогда кролик не исчезнет:

Обратите внимание: событие должно содержать флаг `cancelable: true`. Иначе, вызов `event.preventDefault()` будет проигнорирован.

```
<pre id="rabbit">
  |\  /|
  \|_|/
  /.  .\
  =\_Y_/=
  {>o<}
</pre>

<button onclick="hide()">Hide()</button>

<script>
  // hide() будет вызван при щелчке на кнопке
  function hide() {
    let event = new CustomEvent("hide", {
      cancelable: true // без этого флага preventDefault не работает
    });
    if (!rabbit.dispatchEvent(event)) {
      alert('Действие отменено обработчиком');
    } else {
      rabbit.hidden = true;
    }
  }

  rabbit.addEventListener('hide', function(event) {
    if (confirm("Вызвать preventDefault?")) {
      event.preventDefault();
    }
  });
</script>
```

Вложенные события обрабатываются синхронно

Обычно события обрабатываются асинхронно. То есть, если браузер обрабатывает onclick и в процессе этого произойдёт новое событие, то оно ждёт, пока закончится обработка onclick.

Исключением является ситуация, когда событие инициировано из обработчика другого события.

Тогда управление сначала переходит в обработчик вложенного события и уже после этого возвращается назад.

В примере ниже событие menu-open обрабатывается синхронно во время обработки onclick: **1-> вложенное -> 2**

```
<button id="menu">Меню (нажми меня)</button>

<script>
  menu.onclick = function() {
    alert(1);

    // alert("вложенное событие")
    menu.dispatchEvent(new CustomEvent("menu-open", {
      bubbles: true
    }));

    alert(2);
  };

  document.addEventListener('menu-open', () => alert('вложенное событие'))
</script>
```

Вложенные события обрабатываются синхронно

Обратите внимание, что вложенное событие `menu-open` успевает всплыть и запустить обработчик на `document`. Обработка вложенного события полностью завершается до того, как управление возвращается во внешний код (`onclick`).

Это справедливо не только для `dispatchEvent`, но и для др ситуаций.

Если нам это не подходит, то мы можем либо поместить `dispatchEvent` (или любой другой код, иницирующий события) в конец обработчика `onclick`, либо, если это неудобно, можно обернуть генерацию события в `setTimeout` с нулевой задержкой: **1-> 2-> вложенное**

```
<button id="menu">Меню (нажми меня)</button>

<script>
  menu.onclick = function() {
    alert(1);

    // alert(2)
    setTimeout(() => menu.dispatchEvent(new CustomEvent("menu-open", {
      bubbles: true
    }))), 0);

    alert(2);
  };

  document.addEventListener('menu-open', () => alert('вложенное событие'))
</script>
```

СПАСИБО ЗА ВНИМАНИЕ!

19 февраля 2026 года